CalorieSync — Documentação Técnica

Gerado em 04/11/2026 a partir de docs/.md. A fonte de verdade são os arquivos Markdown do repositório.*

Conteúdo:

1. Telas do aplicativo
2. SPEC.md
3. ARCHITECTURE.md
4. DECISOES.md
5. ADMIN.md
6. TESTING.md
7. MIGRATIONS.md
8. DB_ROLES.md
9. UX_NAVIGATION.md
10. CORES.md
11. EMAIL_SETUP.md
12. HOSPEDAGEM.md
13. ROADMAP.md

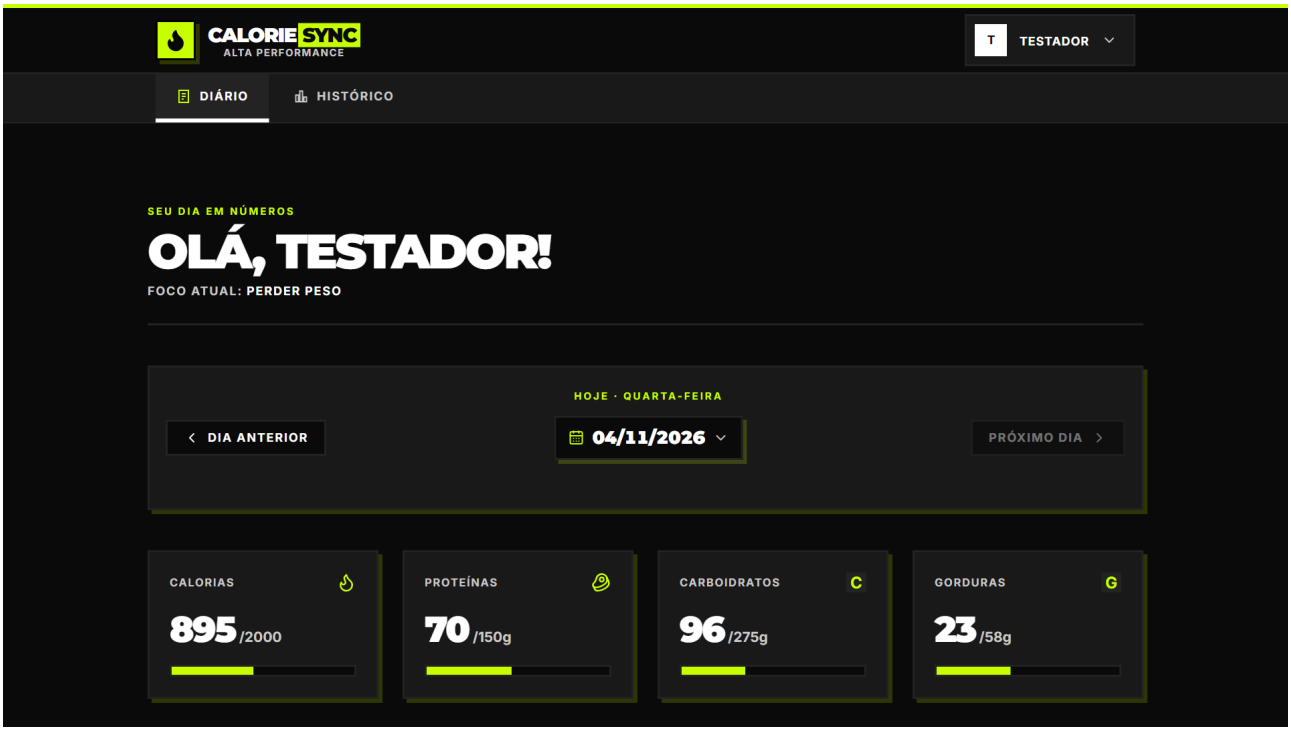
Telas do aplicativo



Página inicial (desktop)



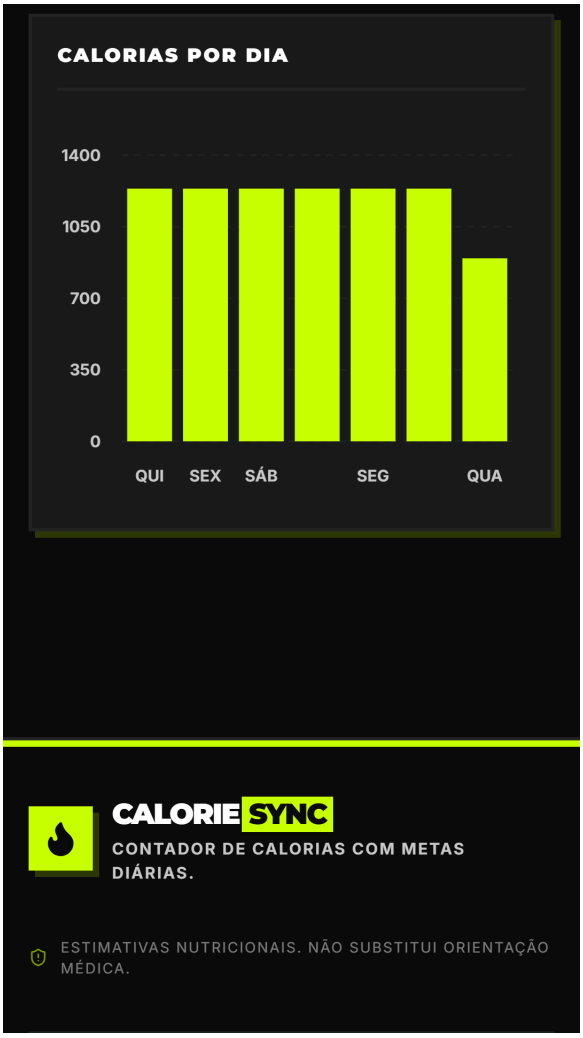
Página inicial (celular)



Diário (desktop)



Diário (celular)



Histórico (celular)



Metas (celular)

Especificação de Requisitos e Regras de Negócio (SPEC.md)

Este documento descreve detalhadamente os Requisitos Funcionais (RFs) e as regras de negócio vigentes no código-fonte do **CalorieSync**. Todas as afirmações foram verificadas diretamente na implementação.

---

1. Requisitos Funcionais (RFs)

RF-001: Autenticação, Confirmação de E-mail, Recuperação de Senha e Sessão [CONFIRMADO]

- **Endpoints públicos de autenticação (`/api/auth/*`, todos com `Cache-Control: no-store`):**
  - `POST /api/auth/register`: Cadastro com nome, e-mail e senha forte. Responde **202** genérico, sem JWT. Conta já verificada não é alterada; o dono recebe um aviso por e-mail.
  - `POST /api/auth/verify-email`: Valida o código de 6 dígitos, marca `email_verified_at` e devolve **200** com `{ user, token }`.
  - `POST /api/auth/resend-code`: Reemite o código para conta existente e não verificada; resposta **202** genérica.
  - `POST /api/auth/login`: Autentica e devolve **200** com `{ user, token }` somente para conta verificada. Senha correta + conta não verificada devolve **403** com `code: "EMAIL_NOT_VERIFIED"` e dispara um novo código (respeitando o cooldown).
  - `POST /api/auth/forgot-password`: Gera link de reset apenas para conta existente e verificada; resposta **202** genérica.
  - `POST /api/auth/reset-password`: Atualiza a senha na mesma transação em que consome o token e incrementa `token_version`; resposta **200** sem sessão.
  - `POST /api/auth/demo`: Cria um visitante temporário (`is_guest`, e-mail `.invalid`, senha aleatória) com perfil, metas e 7 dias de refeições, e devolve **200** com `{ user, token }`. Limite de 20 por IP em 15 min; com 500 visitantes ativos responde **503** `DEMO_UNAVAILABLE`.
- **Endpoints autenticados de conta:**
  - `GET /api/users/me`: Dados cadastrais do usuário logado (sem hash de senha).
  - `PUT /api/users/me`: Atualização de dados cadastrais, biometria e credenciais. Visitantes (`is_guest`) recebem `403` com `code: "DEMO_READ_ONLY"`.
  - `DELETE /api/users/me`: Exclusão definitiva mediante confirmação com a senha atual. Visitantes recebem `403 DEMO_READ_ONLY` (idem nas rotas de MFA).
  - `PUT /api/users/me/onboarding`: Conclusão única do onboarding.
- **Regras de Negócio:**
  - **Criptografia:** Senhas são protegidas com `bcrypt` (fator de custo 10 em desenvolvimento/produção; custo 1 em ambiente de testes para agilidade).

- **Política de Senha Forte (PC-08 / SEC-004):** O cadastro, a redefinição e a alteração de credenciais exigem senha com no mínimo 8 caracteres, no máximo 72 caracteres, e obrigatoriedade de ao menos uma letra e um número (`/(?=.*[a-zA-Z])(?=.*[0-9])/`).
- **Mitigação de Timing Attack no Login (PC-08 / SEC-005):** Caso o e-mail não seja encontrado no banco durante `POST /api/auth/login`, uma verificação `bcrypt.compare` contra um hash dummy pré-computado é executada antes do retorno 401, equalizando o tempo de resposta e prevenindo a enumeração de contas por temporização.
- **Tokens JWT com revogação (A8):** A sessão usa JWT HS256 em cookie HttpOnly `cs_session` (SameSite=Strict, checagem de Origin em mutações; Bearer ainda aceito para clientes de API) assinado com `JWT_SECRET` (mínimo de 32 caracteres), validade configurável por `JWT_EXPIRES_IN` (padrão `1d`) e claim `tv` (`token_version`). O middleware consulta `users.token_version` no banco e rejeita com 401 quando o usuário não existe ou o valor diverge. Troca autenticada de senha ou e-mail incrementa a versão e devolve um token novo.
- **Confirmação de e-mail (BL-12 / A2):** Código de 6 dígitos gerado com `crypto.randomInt`, válido por 15 minutos, com no máximo 5 tentativas por código, uso único e emissão de novo código invalidando os anteriores. Persistência apenas do HMAC-SHA256 com `AUTH_TOKEN_PEPPER`; comparação em tempo constante. Reenvio com cooldown de 60 s e teto de 5 envios por hora por e-mail.
- **Recuperação de senha (BL-05 / A3):** Token de 32 bytes aleatórios em base64url, persistido apenas como SHA-256, válido por 30 minutos e de uso único. O link usa o fragmento `${APP_BASE_URL}/redefinir-senha#token=...`, que não é enviado ao servidor em navegações; o frontend lê o token, limpa a URL com `history.replaceState` e o envia no corpo do POST.
- **Sem enumeração de contas (A4):** `register`, `resend-code` e `forgot-password` respondem sempre o mesmo status e a mesma mensagem, exista ou não a conta. O envio de e-mail ocorre sem `await` após o commit e falhas são registradas sem segredos.
- **Alteração Segura de E-mail ou Senha:** Caso o usuário altere seu e-mail ou defina uma nova senha, o envio da senha atual (`currentPassword`) é mandatório. A troca de e-mail zera `email_verified_at`, incrementa `token_version` e emite novo código de verificação para o endereço novo.
- **Limites de autenticação (A9):** Reset por e-mail: 5/hora; `verify-email`: 10 tentativas/15 min por IP; `reset-password`: 10/hora por IP; reenvio de código: 10/15 min por IP. A chave dos limitadores fica isolada em uma única função.
- **E-mails transacionais (A10):** Interface `Mailer` com `SmtpMailer` (nodemailer) e `MemoryMailer` (testes), escolhida por `MAIL_TRANSPORT`; `NODE_ENV=test` força memória. Proibido registrar código, token, URL de reset ou senha em logs.
- **Exclusão com Cascata:** A exclusão de um usuário dispara remoção física imediata em todas as tabelas dependentes (`meals`, `meal_items`, `body_measurements`, `user_goals`, `email_verification_codes`, `password_reset_tokens`) por integridade referencial `ON DELETE CASCADE`.
- **Execução Única de Onboarding (PC-07 / SEC-008):** O endpoint `PUT /api/users/me/onboarding` só pode ser concluído uma única vez. Se `onboarding_completed === true`, requisições subsequentes são bloqueadas com status `400 Bad Request` ("Onboarding já foi concluído para este usuário."), impedindo

poluição da tabela `body_measurements` e recalculações espúrias. Atualizações posteriores de medidas corporais devem ocorrer via `PUT /api/users/me`.

- **Proteção e Privacidade de Dados de Saúde no Cliente (PC-19 / SEC-018):** O armazenamento local persistente (`localStorage.getItem('user_session')`) é estritamente restrito a metadados de sessão não sensíveis (`id`, `name`, `email`, `onboarding_completed`). Dados de saúde, biometria e metas nutricionais (peso, altura, % de gordura, sexo, data de nascimento, calorias e macronutrientes) são mantidos exclusivamente em memória de execução do cliente React e hidratados via `GET /api/users/me`, impedindo o armazenamento em texto plano de dados sensíveis protegidos pela LGPD. E-mail e código de verificação nunca são gravados em `localStorage`/`sessionStorage`.

RF-002: Cálculo de Taxa Metabólica Basal (TMB), TDEE e Metas Nutricionais [CONFIRMADO]

Implementado de forma pura em `backend/src/services/nutrition.service.ts` e consumido no onboarding e na atualização de metas.

1. Cálculo de Idade

```
age = ano_atual - ano_nascimento
// Ajustado subtraindo 1 caso o aniversário ainda não tenha ocorrido no ano corrente
```

2. Cálculo da Taxa Metabólica Basal (TMB)

O sistema seleciona dinamicamente a equação científica com base na disponibilidade do percentual de gordura corporal (`body_fat`):

- **Opção A: Fórmula de Katch-McArdle (quando `body_fat > 0` informado):** $$\text{Massa Magra (kg)} = \text{peso} \times \left(1 - \frac{\text{body_fat}}{100}\right)$$ $$\text{TMB} = 370 + (21.6 \times \text{Massa Magra})$$
- **Opção B: Fórmula de Mifflin-St Jeor (quando `body_fat` não informado ou nulo):** $$\text{TMB Base} = (10 \times \text{peso em kg}) + (6.25 \times \text{altura em cm}) - (5 \times \text{idade})$$
  - Se `sex === 'masculino'`: $$\text{TMB} = \text{TMB Base} + 5$$
  - Se `sex === 'feminino'` ou `sex === 'prefiro_nao_responder'`: $$\text{TMB} = \text{TMB Base} - 161$$

3. Gasto Energético Total Diário (TDEE)

$$\text{TDEE} = \text{TMB} \times \text{Multiplicador de Atividade}$$

Multiplicadores aplicados:

- `sedentario`: 1.2
- `pouco_ativo`: 1.375

- `moderadamente_ativo`: 1.55
- `muito_ativo`: 1.725
- `extremamente_ativo`: 1.9 *(Caso nível não especificado, aplica o padrão 1.2)*.

4. Calorias Sugeridas por Objetivo

- `perder_peso`: Déficit calórico de 20% ($\text{TDEE} \times 0.80$)
- `ganhar_massa`: Superávit calórico de 15% ($\text{TDEE} \times 1.15$)
- `manter_peso`: Manutenção calórica ($\text{TDEE} \times 1.0$)

5. Piso Mínimo de Segurança Calórica

Para evitar dietas restritivas extremas danosas à saúde:

- Usuários do sexo masculino: Mínimo inegociável de **1500 kcal**.
- Usuárias do sexo feminino ou não respondido: Mínimo inegociável de **1200 kcal**. Se o valor calculado for menor que o piso, o valor sugerido é elevado até o limite do piso.

6. Divisão de Macronutrientes (D-15) [CONFIRMADO]

Regras de negócio de saúde aplicadas exclusivamente para geração de metas automáticas (metas manuais permanecem sob livre escolha do usuário). Valores de projeto definidos para revisão periódica por profissional de nutrição:

1. **Determinação da Massa de Referência:**
    - **Com Percentual de Gordura Conhecido (`body_fat` informado e > 0):** Utiliza a Massa Livre de Gordura (LBM) calculada pela fórmula de Katch-McArdle: $$\text{Massa de Referência} = \text{peso} \times \left(1 - \frac{\text{body_fat}}{100}\right)$$
    - **Sem Percentual de Gordura:**
        - Se Obesidade ($\text{IMC} \ge 30.0$): aplica a fórmula de peso ajustado: $$\text{Peso Ideal (PI)} = 22.5 \times \text{altura (m)}^2$$ $$\text{Massa de Referência (Peso Ajustado)} = \text{PI} + 0.25 \times (\text{peso} - \text{PI})$$
        - Se $\text{IMC} < 30.0$: utiliza o peso corporal atual: $$\text{Massa de Referência} = \text{peso}$$
2. **Cálculo Base de Macronutrientes:**
    - **Proteína:** 1.6 g/kg da massa de referência, limitada a no máximo 35% das calorias totais.
    - **Gordura:** 25% das calorias totais (9 kcal/g).
    - **Carboidrato:** Saldo restante das calorias totais (4 kcal/g).
3. **Ordem de Ajuste para Garantia de Carboidratos (se Carboidrato $< 20\%$ das calorias):**
    - Caso o carboidrato calculado resulte em menos de 20% das calorias totais da meta:
        1. Reduzir a gordura progressivamente até o piso de 20% das calorias totais.
        2. Se o carboidrato ainda permanecer abaixo de 20%, reduzir a proteína progressivamente até o piso de 1.2 g/kg da massa de referência.

4. **Invariantes e Reconciliação:**
    - **Tolerância Calórica:** A soma calórica dos macronutrientes finais fica dentro de $\pm 1\%$ da meta estipulada.
    - **Não-negatividade:** Nenhum macronutriente pode ter valor negativo.
    - **Proteção Metabólica:** O carboidrato nunca é forçado a zero por sobrecarga de proteínas e gorduras.
    - **Reconciliação Inteira:** O saldo calórico após proteína e gordura é convertido em gramas inteiras de carboidrato via $\text{round}\left(\frac{\text{kcal} - (\text{P} \times 4) - (\text{G} \times 9)}{4}\right)$, garantindo que a soma aritmética coincida exatamente com as calorias totais exibidas.

RF-003: Vigência de Metas e Histórico Corporal [CONFIRMADO]

- **Endpoints:**
    - `POST /api/users/onboarding`: Conclusão do cadastro inicial, gerando a primeira meta automática e primeira medição corporal.
    - `GET /api/users/goals/active?date=YYYY-MM-DD`: Retorna a meta calórica e de macros que vigora em determinada data.
    - `POST /api/users/goals/preview`: Simula os novos cálculos com base em dados provisórios sem persistir no banco.
    - `PUT /api/users/goals`: Persiste novas metas (modo `auto` ou `manual`).
- **Regras de Negócio:**
    - **Vigência por Data (`effective_from`):** Metas são registradas na tabela `user_goals` com a chave `(user_id, effective_from)`.
    - **Preservação do Histórico:** Ao alterar metas hoje, as refeições de ontem continuam comparadas com a meta vigente ontem. A consulta ativa executa:

      ```
      SELECT * FROM user_goals WHERE user_id = $1 AND effective_from <= $2 ORDER BY
      effective_from DESC LIMIT 1;
      ```

    - **Unicidade Diária (Upsert):** Há no máximo 1 registro de meta por usuário por dia civil; alterações no mesmo dia atualizam o registro existente via `ON CONFLICT (user_id, effective_from) DO UPDATE`.
    - **Histórico de Medições:** A cada conclusão de onboarding ou alteração de peso/gordura no perfil, um registro correspondente é inserido em `body_measurements` com timestamp de medição.

RF-004: Catálogo e Busca de Alimentos [CONFIRMADO]

- **Endpoints:**
    - `GET /api/foods?search=termo&limit=50&offset=0`: Busca de alimentos ativos no catálogo.
- **Regras de Negócio:**
    - **Parâmetro Único de Busca (D-09):** Aceita estritamente o parâmetro `search` (parâmetro alternativo `q` descontinuado). Validado com Zod (limite máximo de caracteres) e

caracteres curinga (`%`, `_`, `\`) devidamente escapados antes da interpolação no SQL `ILIKE`.
- **Busca Tolerante:** Termos com ou sem acentos (ex: "feijao" vs "feijão") e diferenças de maiúsculas/minúsculas encontram os mesmos itens através de `f_unaccent(name) ILIKE f_unaccent($1)`.
- **Ordenação por Similaridade:** O operador `<->` do `pg_trgm` classifica no topo os itens com maior proximidade ortográfica em relação ao que foi digitado.
- **Contrato Público e Projeção Explícita (D-08):** Proibido `SELECT *`. A consulta projeta apenas os campos consumidos pelo cliente: `id`, `name`, `calories`, `proteins`, `carbs`, `fats`, `base_unit`, `serving_weight`, `unit_name`, `unit_weight`, `source`. Campos internos como `external_id` e `active` nunca são expostos na API.
- **Cache HTTP (D-07):** Endpoint autenticado responde com headers `Cache-Control: private, max-age=300`, `ETag` (nativo do Express) e `Vary: Authorization`. Não utiliza cache público nem LRU global na memória do processo.
- **Catálogo Somente-Leitura:** Não existem rotas de criação ou exclusão de alimentos para usuários finais. A base é mantida por migrações e scripts de importação de tabelas como a TACO (`backend/scripts/import-foods.ts`).
- **Ativação Lógica:** Apenas registros com `active = true` são retornados nas consultas.

RF-005: Registro e Gestão de Refeições [CONFIRMADO]

- **Endpoints:**
  - `POST /api/meals`: Registro de nova refeição com múltiplos alimentos.
  - `GET /api/meals/today?date=YYYY-MM-DD`: Listagem das refeições registradas no dia especificado.
  - `PUT /api/meals/:id`: Atualização completa do nome, horário e itens de uma refeição.
  - `DELETE /api/meals/:id`: Exclusão de refeição.
- **Regras de Negócio:**
  - **Fuso Horário Oficial e Janela Diária (D-01):**
    - O servidor é a autoridade única do fuso `America/Sao_Paulo` (`APP_TIMEZONE = 'America/Sao_Paulo'`).
    - Parâmetro `?date=YYYY-MM-DD` é validado via Zod (formato `YYYY-MM-DD`, data de calendário real entre `2000-01-01` e `hoje + 1 dia`). Na ausência do parâmetro, o servidor utiliza `todayInAppTz()` baseado em `Intl.DateTimeFormat('en-CA', { timeZone })`.
    - Datas de calendário são tratadas exclusivamente como strings (proibido `toISOString().split('T')[0]` e `new Date('YYYY-MM-DD')`).
    - A janela diária no SQL utiliza faixa em `meal_time` compatível com índices: `meal_time >= ($1::date)::timestamp AT TIME ZONE $tz AND meal_time < (($1::date + 1))::timestamp AT TIME ZONE $tz`
  - **Cálculo Nutricional Proporcional:** A base nutricional é definida por 100g ou 100ml (`serving_weight = 100`). Se o item utilizar unidade (`un`) e o alimento possuir `unit_weight`, a quantidade em gramas é calculada como $\text{displayAmount} \times \text{unit_weight}$. O fator multiplicador aplicado é: $$\text{fator} = \frac{\text{quantidade

final em gramas}}{\text{serving_weight}}$$ Cada macro do item é arredondado para uma casa decimal: $\text{Math.round}(\text{valor} \times \text{fator} \times 10) / 10$.
- **Imutabilidade Histórica (Denormalização):** Os macros calculados são gravados diretamente nas colunas `calories`, `proteins`, `carbs` e `fats` de `meal_items`. Alterações futuras no alimento base não retroagem no diário do usuário.
- **Atomicidade Transacional:** Toda escrita de refeição abre uma transação (`BEGIN`), grava o cabeçalho em `meals`, insere/substitui os itens em `meal_items` e encerra com `COMMIT` (ou `ROLLBACK` em caso de falha).

RF-006: Estatísticas e Médias Analíticas [CONFIRMADO]

- **Endpoints:**
  - `GET /api/meals/stats?period=week|month&date=YYYY-MM-DD`: Agrupamento histórico para a página de estatísticas.
- **Regras de Negócio:**
  - **Fuso Horário no Agrupamento (D-01):** O agrupamento diário utiliza `(meal_time AT TIME ZONE $tz)::date`. É proibido o uso de `DATE(meal_time)` puro para evitar deslocamento de datas em registros noturnos.
  - **Períodos Suportados:** `week` (últimos 7 dias) ou `month` (últimos 30 dias).
  - **Agregação SQL com `LATERAL JOIN`:** O banco de dados agrupa as refeições por data e correlaciona a meta que estava em vigor em cada um dos dias do intervalo.
  - **Cálculo Justo de Médias:** As médias do período (`averages.consumed` e `averages.goal`) são calculadas dividindo a soma dos macros **exclusivamente pela quantidade de dias em que houve registros efetivos**, sem derrubar as médias artificialmente em virtude de dias em que o usuário não registrou consumo.
  - **Preenchimento de Lacunas:** Para visualização correta nos gráficos, dias do período sem qualquer registro são retornados com zeros em calorias e macronutrientes.

RF-007: Painel Administrativo do Catálogo, Curadoria e RBAC (D-22 / B1–B12) [CONFIRMADO]

- **Endpoints Administrativos (`/api/admin/*`, protegidos por `router.use(authenticate, requireAdmin)`):**
  - `GET /api/admin/foods` (alias `/api/admin/alimentos`): Listagem paginada (`page`, `pageSize=25|50|100`, `q/search`, `status=todos|ativo|inativo`, `sort=nome|kcal|status`, `order=asc|desc`) com ordenação determinística por `id ASC`, escape de `ILIKE` e retorno estruturado `{ data, meta: { page, pageSize, total, totalPages } }`.
  - `POST /api/admin/foods`: Criação de alimento global (`user_id = NULL`), com validação estrita por 100 g/ml (`calories <= 900`, macros `<= 100`, `P+C+G <= 100.5 g`), exigência de `source_ref` oficial auditável e warnings informativos para divergência calórica > 25%.
  - `PUT /api/admin/foods/:id`: Edição de dados nutricionais e apresentação de alimentos globais. Marca `admin_edited_at = NOW()`.

    - `PATCH /api/admin/foods/:id/status`: Alternância de status ativo/inativo (`active: boolean`). Sem rota DELETE de alimentos.
- **Regras de Negócio Administrativas:**
    - **Autorização Dinâmica por PK (B1):** O papel `role` é consultado no banco por chave primária a cada requisição no middleware `authenticate`. É proibido autorizar por claims do JWT.
    - **Sem Contas Pré-definidas (B2):** Nenhum administrador é criado via migração, seed ou variável de ambiente. A promoção é realizada exclusivamente via CLI (`npm run admin:promote <email>`) após cadastro e e-mail verificado.
    - **Restrição de Escopo (B3):** O painel administrativo gerencia unicamente o catálogo global. Nenhum endpoint administrativo lê ou gerencia refeições, perfis, metas ou lista de usuários.
    - **Auditoria Transacional Atômica (B5):** Toda operação de escrita administrativa grava na mesma transação em `admin_audit_log` (`actor_type: 'user' | 'cli'`).
    - **Integridade de Refeições com Inativos (B7):** Novos lançamentos de refeição com alimento inativo são rejeitados com status `422 Unprocessable Entity`.
    - **Proteção da Importação TACO (B10):** A rotina `import-foods.ts` nunca reativa alimentos desativados (`active = foods.active`) e nunca sobrescreve alimentos curados (`admin_edited_at IS NOT NULL`).

RF-008: Navegação, Telas e Gesto de Swipe [CONFIRMADO]

- **Destinos (fonte única `frontend/src/navigation/nav-items.ts`):** Diário (`/`, `/diario/:date`), Histórico (`/historico/*`), Conta (`/conta`, `/metas/*`, `/perfil`, `/configuracoes/*`). Barra inferior no mobile (`md:hidden`): `[DIÁRIO] [HISTÓRICO] [+ NOVA] [CONTA]`; o "+" é uma ação (modal global de Nova Refeição), não uma rota.
- **Tela Diário:** antiga "Dashboard"; subtítulo "SEU DIA EM NÚMEROS"; rotas inalteradas.
- **Tela `/conta`:** cartão do usuário (iniciais, nome, e-mail), atalhos para Metas, Perfil, Configurações (e Catálogo, se admin), alternador de tema e "Sair da conta".
- **Submenus:** componente único `SegmentedNav` (`<nav aria-label>` com links, `aria-current="page"` na aba ativa).
- **Swipe entre os destinos da barra inferior (só mobile, só toque):** `useSwipeNav` (`frontend/src/navigation/useSwipeNav.ts`) em `SwipeMain` (o `<main>` do `RootLayout`), na ordem de `BOTTOM_NAV_ITEMS` (Diário → Histórico → Conta), sem wrap-around; dedo para a esquerda = próximo, direita = anterior. Só vale em rotas que pertencem a um destino da barra. `classifySwipe` (`swipe.ts`) decide ao soltar com `SWIPE_CONFIG`: `|dx| ≥ 2 × |dy|`, distância ≥ max(64 px, 30% da largura) ou flick ≥ 40 px a ≥ 0,5 px/ms; toque inicial a menos de 24 px das bordas é ignorado. Durante o arrasto (após 10 px, se horizontal) o painel de conteúdo acompanha o dedo com `translateX` e opacidade, com resistência (×0,25) na direção sem destino; ao confirmar, sai em 180 ms e a nova tela entra em 220 ms pelo lado oposto; se não confirmar, volta em 220 ms. Com `prefers-reduced-motion`, navega direto. O gesto é ignorado em `[data-no-swipe]`, sliders, campos, `[role=dialog|listbox|menu|combobox]`, gráficos Recharts, ancestrais com rolagem horizontal real, com dois dedos, texto selecionado, campo em foco, overlay aberto ou `touchcancel`. Listeners nativos e passivos no `<main>` (sem `preventDefault`); o `transform` só existe durante o gesto. **Não há swipe entre abas irmãs.**

2. Tratamento Centralizado de Erros e Contratos de API (D-10) [CONFIRMADO]

1. **Contrato Universal de Resposta:** Toda resposta HTTP da API segue estritamente:

```
{
  "success": boolean,
  "data"?: any,
  "message"?: string,
  "code"?: string
}
```

   `code` é opcional e carrega códigos de negócio estáveis consumidos pelo cliente, como `EMAIL_NOT_VERIFIED`, `INVALID_VERIFICATION_CODE`, `INVALID_RESET_TOKEN` e `USER_NOT_FOUND`. Códigos HTTP adicionais do fluxo de autenticação: `202 Accepted` para respostas genéricas sem corpo de sessão e `403 Forbidden` para credencial válida com e-mail ainda não confirmado.
2. **Hierarquia de Erros Tipados (`AppError`):**
   - Classes dedicadas estendendo `AppError`: `ValidationError` (400), `UnauthorizedError` (401), `NotFoundError` (404), `ConflictError` (409).
   - Encaminhamento assíncrono via `asyncHandler` para o Express 4 delegar ao `next(err)`.
3. **Tratamento Centralizado e Mascaramento de 500:**
   - Erros desconhecidos respondem status `500` com mensagem genérica opaca (`"Erro interno do servidor"`).
   - Proibido expor `err.message` ou stack traces em respostas JSON. Detalhes reais são registrados unicamente em `console.error` no servidor.
   - Códigos de erro do PostgreSQL são mapeados sem vazamento de strings cruas: `23505` (unique violation) → 409, `23503` (foreign key) → 409, `22P02` (invalid syntax) → 400, demais → 500.

Arquitetura do Sistema (ARCHITECTURE.md)

Este documento detalha o desenho arquitetural, o modelo relacional de dados, as camadas de software e as decisões técnicas implementadas no **CalorieSync**.

---

1. Visão Geral e Camadas [CONFIRMADO]

O projeto é estruturado em um monorepo com separação estrita entre o cliente SPA e o servidor de API:

```
[Cliente Web (React 19 + Vite)]
         │
         ▼  (HTTP REST / JSON - Authorization: Bearer <JWT>)
[Middlewares (CORS, Helmet, Rate-Limit, Auth JWT, Zod Validate)]
         │
         ▼
[Controllers (Express - Parse HTTP & Resposta Padronizada)]
         │
         ▼
[Services (Regras de Negócio Puras, TMB, Transações)]
         │
         ▼
[Models (SQL Puro via pg.Pool - Parametrizado)]
         │
         ▼
[(PostgreSQL 14+ com pg_trgm e unaccent)]
```

Camadas do Backend (`/backend/src/`) [CONFIRMADO]

1. **`routes/`**: Declaração dos endpoints e associação com middlewares específicos (`authenticate`, `validate(schema)`), incluindo probe de integridade do banco `/health`.
2. **`controllers/`**: Extração de dados da requisição (`req.body`, `req.params`, `req.query`, `req.userId`), invocação do service correspondente e devolução de resposta formatada no padrão `{ success, data, message }`, com headers de cache privado (`Cache-Control`, `Vary: Authorization`) no catálogo de alimentos.
3. **`services/`**: Lógica de aplicação, cálculos matemáticos (TMB/TDEE, divisão proporcional de macros), coordenação de persistência, transações atômicas (`BEGIN`/`COMMIT`/`ROLLBACK`) e validações de regras de negócio.
4. **`models/`**: Consultas diretas ao PostgreSQL via driver nativo `pg.Pool`, com parametrização estrita de valores contra SQL Injection, projeções explícitas de colunas e inserções multi-row em lote.
5. **`middlewares/`**:
   - `compression`: Compressão HTTP ativa de respostas no topo da cadeia.
   - `auth.ts`: Validação de tokens JWT, extração de `userId`, consulta de revogação no banco via `token_version` e carregamento de `role` por chave primária (`req.userRole`).
   - `requireAdmin`: Middleware de autorização RBAC que exige `req.userRole === 'admin'`, rejeitando usuários comuns com 403 Forbidden.
   - `authRateLimit.ts`: Limitadores das rotas `/api/auth/*`, com a chave isolada em uma única função.

- `validate.ts`: Middleware genérico para validação particionada de esquemas Zod (body, params, query) retornando 400 em caso de falha.
- `requestLogger.ts`: Logging HTTP estruturado em JSON e propagação/geração do header `X-Request-Id` (correlation ID) para rastreabilidade de ponta a ponta.
- `errorHandler.ts`: Captura global de exceções não tratadas com resposta 500 mascarada sem vazamento de detalhes internos.

Camadas do Frontend (`/frontend/src/`) [CONFIRMADO]

1. **`router.tsx` (`react-router` v7)**: Roteamento SPA declarativo via `createBrowserRouter`, suporte a URLs amigáveis, rotas públicas/protegidas e code-splitting por rota com `React.lazy()`.
2. **`services/api.ts` (`apiFetch`)**: Wrapper centralizado de `fetch` que injeta o token Bearer, trata respostas não-JSON (502 HTML) defensivamente, propaga `ApiError` com `status`/`code` e sincroniza o evento global `auth:unauthorized` sob 401.
3. **TanStack Query (React Query v5)**: Gerencia chamadas assíncronas, estado de cache, revalidação e invalidação cruzada de dados (`mealStats`, `todayMeals`, `active-goal`).
4. **`context/AuthContext.tsx`**: Gerencia autenticação global, sincronização do token e perfil em memória, mantendo o `localStorage` sanitizado sem dados biométricos.
5. **`components/` & `pages/`**: Componentes estilizados com Tailwind CSS em estética brutalista (`rounded-none`, bordas contrastantes) e modais acessíveis via Radix UI com focus trap.
6. **`navigation/` (fonte única de navegação):** `nav-items.ts` guarda destinos, abas, ícones (contorno e preenchido, em `icons.tsx`) e a regra de rota ativa; consumido por `BottomNav`, `AppHeader`, `AccountPage` e `SegmentedNav`. Também abriga o swipe entre destinos da barra inferior com arrasto animado (`swipe.ts`, `useSwipeNav.ts`, `SwipeMain.tsx`, usado pelo `<main>` do `RootLayout`) e `useHideOnScroll.ts`.
7. **Hosts globais no `RootLayout`:** `ThemeProvider` (tema claro/escuro, `useTheme`) e `NewMealModalProvider` (`useNewMealModal().open({ date?, meal? })`), que renderiza o `NewMealModal` lazy uma única vez e mantém a invalidação de queries do próprio modal.

Pacote Compartilhado (`/shared/`) [CONFIRMADO]

1. **Fonte única de contratos**: schemas Zod puros (`src/schemas/`), DTOs/tipos (`src/types/`) e cálculo nutricional (`src/nutrition/nutrients.ts`) consumidos por backend e frontend.
2. **Resolução por alias**: `@caloriesync/shared` (barrel) e `@caloriesync/shared/nutrition` (subpath do cálculo) configurados em `backend/tsconfig.json`, `frontend/tsconfig.app.json` e `frontend/vite.config.ts`. Sem `npm workspaces` nesta branch.
3. **Bundle enxuto**: imports de valores usam o subpath `nutrition` para não arrastar os schemas Zod ao bundle do frontend; imports apenas de tipos usam o barrel e são apagados na compilação.

Fluxo de autenticação por e-mail (D-21 / BL-12 / BL-05) [CONFIRMADO]

1. **Cadastro:** `POST /api/auth/register` cria a conta com `email_verified_at = NULL` e responde 202 genérico. Um código de 6 dígitos é persistido apenas como HMAC-SHA256 e enviado após o commit. Nenhuma sessão é emitida.

2. **Verificação:** `POST /api/auth/verify-email` valida o código em transação (expiração, 5 tentativas, uso único), marca `email_verified_at` e emite JWT com `tv`.
3. **Login:** `POST /api/auth/login` aplica o hard gate: conta não verificada recebe 403 `EMAIL_NOT_VERIFIED`, nunca JWT. O código é reemitido respeitando cooldown e teto por e-mail.
4. **Recuperação:** `POST /api/auth/forgot-password` só gera token para conta verificada. O token de 32 bytes é persistido como SHA-256, enviado no fragmento `#token=...` e consumido em `POST /api/auth/reset-password`, que atualiza a senha e incrementa `token_version` na mesma transação, sem auto-login.
5. **Revogação:** o middleware compara `tv` com `users.token_version`; troca de senha, troca de e-mail e reset invalidam tokens antigos. O frontend persiste o token renovado devolvido por `PUT /api/users/me`.
6. **Entrega:** `Mailer` plugável (`SmtpMailer`/`MemoryMailer`) escolhido por `MAIL_TRANSPORT`. Mailpit captura mensagens em desenvolvimento (SMTP 1025 / UI 8025). Códigos, tokens e URLs de reset nunca são logados.

---

2. Diagrama Entidade-Relacionamento (ER) [CONFIRMADO]

O esquema do banco de dados é composto por **9 tabelas**, geradas pelas migrações em `backend/migrations/`:

```
erDiagram
    users ||--o{ meals : "possui"
    users ||--o{ body_measurements : "registra"
    users ||--o{ user_goals : "possui vigencia"
    users ||--o{ email_verification_codes : "recebe codigos"
    users ||--o{ password_reset_tokens : "solicita resets"
    users ||--o{ admin_audit_log : "ator em"
    meals ||--o{ meal_items : "contem"
    foods ||--o{ meal_items : "referenciado em"

    users {
        UUID id PK "default uuid_generate_v4()"
        VARCHAR name "NOT NULL"
        VARCHAR email UK "NOT NULL, UNIQUE"
        VARCHAR password_hash "NOT NULL"
        VARCHAR role "NOT NULL, DEFAULT 'user', CHECK (role IN ('user', 'admin'))"
        VARCHAR goal "NOT NULL, DEFAULT 'perder_peso'"
        VARCHAR sex "NULLABLE"
        DATE birth_date "NULLABLE"
        NUMERIC weight "5,2 NULLABLE"
        NUMERIC height "5,2 NULLABLE"
        NUMERIC body_fat "5,2 NULLABLE"
        VARCHAR activity_level "NULLABLE"
        NUMERIC daily_calories_goal "6,2 DEFAULT 2000"
        NUMERIC daily_proteins_goal "6,2 DEFAULT 100"
        BOOLEAN onboarding_completed "DEFAULT FALSE"
        TIMESTAMPTZ email_verified_at "NULLABLE"
        INTEGER token_version "NOT NULL, DEFAULT 0"
        BOOLEAN is_guest "NOT NULL, DEFAULT FALSE"
        TIMESTAMPTZ terms_accepted_at "NULLABLE"
        TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
        TIMESTAMPTZ updated_at "DEFAULT CURRENT_TIMESTAMP"
    }

    foods {
        UUID id PK "default uuid_generate_v4()"
        VARCHAR name "NOT NULL"
        NUMERIC calories "8,2 NOT NULL"
        NUMERIC proteins "8,2 NOT NULL"
        NUMERIC carbs "8,2 NOT NULL"
        NUMERIC fats "8,2 NOT NULL"
        VARCHAR base_unit "10 NOT NULL, DEFAULT 'g'"
        NUMERIC serving_weight "8,2 NOT NULL, DEFAULT 100"
        VARCHAR unit_name "50 NULLABLE"
        NUMERIC unit_weight "8,2 NULLABLE"
        VARCHAR source "50 NULLABLE"
        VARCHAR source_ref "255 NULLABLE"
        VARCHAR external_id "100 NULLABLE"
        BOOLEAN active "NOT NULL, DEFAULT TRUE"
        TIMESTAMPTZ admin_edited_at "NULLABLE"
        TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
        TIMESTAMPTZ updated_at "DEFAULT CURRENT_TIMESTAMP"
    }

    admin_audit_log {
        UUID id PK "default uuid_generate_v4()"
        UUID actor_user_id FK "NULLABLE REFERENCES users(id)"
        VARCHAR actor_type "NOT NULL, CHECK (actor_type IN ('user', 'cli'))"
        VARCHAR action "NOT NULL"
        VARCHAR entity_type "NOT NULL"
        VARCHAR entity_id "NOT NULL"
        JSONB before "NULLABLE"
        JSONB after "NULLABLE"
        TIMESTAMPTZ created_at "NOT NULL, DEFAULT CURRENT_TIMESTAMP"
    }

    meals {
        UUID id PK "default uuid_generate_v4()"
        UUID user_id FK "NOT NULL, ON DELETE CASCADE"
        VARCHAR name "NOT NULL"
        TIMESTAMPTZ meal_time "NOT NULL"
        TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
        TIMESTAMPTZ updated_at "DEFAULT CURRENT_TIMESTAMP"
    }
```

```
meal_items {
    UUID id PK "default uuid_generate_v4()"
    UUID meal_id FK "NOT NULL, ON DELETE CASCADE"
    UUID food_id FK "NOT NULL, ON DELETE RESTRICT"
    NUMERIC quantity "8,2 NOT NULL"
    VARCHAR unit "10 NOT NULL, DEFAULT 'g'"
    NUMERIC display_amount "8,2 NULLABLE"
    NUMERIC calories "8,2 NOT NULL"
    NUMERIC proteins "8,2 NOT NULL"
    NUMERIC carbs "8,2 NOT NULL"
    NUMERIC fats "8,2 NOT NULL"
    TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
}

body_measurements {
    UUID id PK "default uuid_generate_v4()"
    UUID user_id FK "NOT NULL, ON DELETE CASCADE"
    NUMERIC weight "5,2 NOT NULL"
    NUMERIC body_fat "5,2 NULLABLE"
    TIMESTAMPTZ measured_at "NOT NULL, DEFAULT CURRENT_TIMESTAMP"
    TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
}

user_goals {
    UUID id PK "default uuid_generate_v4()"
    UUID user_id FK "NOT NULL, ON DELETE CASCADE"
    DATE effective_from "NOT NULL"
    NUMERIC calories "6,2 NOT NULL"
    NUMERIC proteins "6,2 NOT NULL"
    NUMERIC carbs "6,2 NOT NULL"
    NUMERIC fats "6,2 NOT NULL"
    VARCHAR mode "20 NOT NULL, DEFAULT 'auto'"
    TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
}

email_verification_codes {
    UUID id PK "default gen_random_uuid()"
    UUID user_id FK "NOT NULL, ON DELETE CASCADE"
    TEXT code_hash "HMAC-SHA256 do codigo"
    TIMESTAMPTZ expires_at "NOT NULL"
    INTEGER attempts "NOT NULL, DEFAULT 0"
    TIMESTAMPTZ consumed_at "NULLABLE"
    TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
}

password_reset_tokens {
    UUID id PK "default gen_random_uuid()"
    UUID user_id FK "NOT NULL, ON DELETE CASCADE"
    TEXT token_hash "SHA-256, UNIQUE"
    TIMESTAMPTZ expires_at "NOT NULL"
    TIMESTAMPTZ used_at "NULLABLE"
    TIMESTAMPTZ created_at "DEFAULT CURRENT_TIMESTAMP"
}
```

Índices e Constraints Críticas [CONFIRMADO]

- `users`:
  - `users_email_lower_key`: índice único em `lower(email)` (migração `1792600000000_add-users-email-lower-index.js`).
  - `email_verified_at` e `token_version` adicionados pela migração de BL-12/BL-05.
  - `is_guest` e índice parcial `users_guest_created_at_idx` em `(created_at) WHERE is_guest` (migração `1793600000000_add-guest-users.js`), usados pela limpeza de visitantes.
- `email_verification_codes`:

    - Índice `ON email_verification_codes(user_id, created_at DESC)` para localizar o código mais recente e contar envios por janela.
- `password_reset_tokens`:
    - `token_hash` único (busca direta do token apresentado) e índice `ON password_reset_tokens(user_id, created_at DESC)` para contar pedidos por janela.
- `foods`:
    - `foods_source_external_id_key`: Constraint de unicidade composta `UNIQUE (source, external_id)`.
    - `idx_foods_name_trgm`: Índice GIN de similaridade trigram: `CREATE INDEX idx_foods_name_trgm ON foods USING GIN (f_unaccent(name) gin_trgm_ops)`.
    - `idx_foods_active_name`: Índice parcial B-tree: `CREATE INDEX idx_foods_active_name ON foods (name) WHERE active` (migração `1792099463766_add-food-and-meal-indexes.js` / PC-10).
    - `idx_foods_active_unique_name`: Índice único parcial B-tree: `CREATE UNIQUE INDEX idx_foods_active_unique_name ON foods (f_unaccent(lower(name))) WHERE active = true` (migração `1792445063767_add-food-name-uniqueness.js` / BL-09).
- `meals`:
    - `idx_meals_user_id_meal_time`: Índice composto `ON meals(user_id, meal_time DESC)` para aceleração de buscas no Diário.
- `meal_items`:
    - `idx_meal_items_meal_id`: Índice simples `ON meal_items(meal_id)`.
    - `idx_meal_items_food_id`: Índice de foreign key: `CREATE INDEX idx_meal_items_food_id ON meal_items (food_id)` (migração `1792099463766_add-food-and-meal-indexes.js` / PC-10).
- `body_measurements`:
    - Índice composto `ON body_measurements(user_id, measured_at)`.
- `user_goals`:
    - `unique_user_goal_per_day`: Constraint de unicidade `UNIQUE (user_id, effective_from)`, garantindo uma única meta vigente por data para cada usuário.

---

3. Decisões Técnicas com Justificativa [CONFIRMADO]

Decisão	Escolha	Justificativa
Acesso a Dados	Driver nativo `pg` (sem ORM)	Elimina a sobrecarga e abstração de ORMs (Prisma, Sequelize), permitindo consultas analíticas com SQL puro (CTEs, `LATERAL JOIN`, extensões de trigram).
Migrações	`node-pg-migrate` com ESM nativo	Gerenciamento determinístico de alterações de esquema em arquivos JS/SQL versionados no Git, sem estado embutido obscuro. Migrações aplicadas são imutáveis (D-13): novas migrações usam `export const up/down`; as legadas em CommonJS ficam congeladas (BL-15, ver `backend/migrations/README.md`).

Decisão	Escolha	Justificativa
Denormalização Nutricional	Macros gravados em `meal_items`	Quando uma refeição é criada, as calorias e macronutrientes da porção são calculados e persistidos diretamente no registro do item. Se os dados do alimento base forem alterados ou removidos futuramente, o registro histórico do que o usuário efetivamente consumiu naquele dia permanece íntegro e imutável.
Transações Atômicas	`BEGIN` / `COMMIT` / `ROLLBACK`	Operações de gravação que tocam `meals` e múltiplos `meal_items` (criação e atualização) executam obrigatoriamente dentro de transações de banco para evitar registros órfãos.
Validação Universal	Zod em todo o pipeline	O Zod é usado para validar variáveis de ambiente na inicialização (fail-fast), parâmetros HTTP nos middlewares do backend e entradas de formulários no frontend.
Autenticação por e-mail (D-21)	Código OTP com HMAC-SHA256, reset com SHA-256, hard gate no login e `token_version`	Evita credenciais em texto puro, impede sessão para e-mail não comprovado, permite revogação de tokens e mantém respostas sem enumeração de contas. O token de reset viaja no fragmento da URL e nunca em logs de servidor.
E-mail transacional (D-21/A10)	`Mailer` plugável (`SmtpMailer`/`MemoryMailer`) com escolha por env	Desacopla o código do provedor (sem SDK), permite Mailpit em desenvolvimento e caixa em memória nos testes, com o mesmo contrato de envio.
Catálogo Protegido	`foods` somente-leitura para usuários	Usuários comuns não possuem endpoints para cadastrar, editar ou apagar alimentos globais, evitando corrupção do catálogo. A inserção é restrita a migrações e scripts de carga (`import-foods.ts`). Um índice único parcial (`f_unaccent(lower(name))` onde `active = true`, BL-09) garante a idempotência do seed sem duplicar nomes.
Estratégia de Fuso Horário (D-01)	Opção A+B: servidor fixa `America/Sao_Paulo` via `Intl` e consultas com `AT TIME ZONE`; cliente envia `?date=YYYY-MM-DD`	Desacopla a consistência das datas civis do fuso do ambiente operacional e garante agregação correta independente de onde o servidor ou container esteja executando.
Tratamento de Falhas 500 (D-10)	Mascaramento de erros internos nos controllers com mensagem genérica	Previne vazamento de detalhes estruturais, tabelas ou constraints do PostgreSQL em respostas HTTP 500.
SSL e Proxy Seguro (D-19)	SSL estrito no pool em prod (`rejectUnauthorized: true`), CA via `DB_SSL_CA`, e proxy via `TRUST_PROXY`	Evita ataques de man-in-the-middle em bancos remotos mantendo flexibilidade para certificados de nuvem e proxies reversos.

---

4. Tratamento de Fuso Horário e Datas [CONFIRMADO]

1. **Definição e Desacoplamento de Fuso no Node.js (D-01):**
    - Embora scripts injetem `cross-env TZ=America/Sao_Paulo`, as rotas e regras de negócio não dependem do fuso da máquina hospedeira: manipulações de data civil e

janelas diárias utilizam `getSaoPauloDateString` ancorado em `Intl.DateTimeFormat('en-CA', { timeZone: 'America/Sao_Paulo' })` e queries de banco utilizam `AT TIME ZONE 'America/Sao_Paulo'`.

2. **Armazenamento no Banco de Dados:**
   - Colunas de momento temporal (`created_at`, `updated_at`, `meal_time`, `measured_at`) são tipadas como `TIMESTAMP WITH TIME ZONE` (`TIMESTAMPTZ`), garantindo persistência em UTC com preservação de offset.
   - Colunas puramente de data civil (`birth_date`, `user_goals.effective_from`) são do tipo `DATE`.
3. **Consultas Diárias:**
   - Na listagem de refeições de hoje (`GET /api/meals/today`) e estatísticas (`GET /api/meals/stats`), o frontend envia o parâmetro de consulta `date=YYYY-MM-DD` com base no relógio do navegador. O backend utiliza esse intervalo `[dateT00:00:00-03:00, date+1T00:00:00-03:00[` para evitar que usuários no fuso UTC-3 vejam refeições agrupadas no dia incorreto.

---

5. Estratégia de Busca em Escala (`pg_trgm` + `unaccent`) [CONFIRMADO]

Para viabilizar a pesquisa em catálogos com centenas ou milhares de alimentos (ex: tabela TACO) sem lentidão e tolerando erros ortográficos e acentuação:

1. **Extensões Ativas:**
   - `pg_trgm`: Decompõe textos em trigramas de 3 caracteres para cálculo de similaridade.
   - `unaccent`: Remove diacríticos e acentos do português (ex: "maçã" vira "maca").
2. **Wrapper Imutável (`f_unaccent`):**
   - O PostgreSQL exige que funções usadas em índices de expressão sejam estritamente `IMMUTABLE`. Como `unaccent` nativo depende de dicionários locais, a migração `1791390511650_add-food-scale-features.js` cria uma função wrapper declarada explicitamente como `IMMUTABLE`:

```
CREATE OR REPLACE FUNCTION f_unaccent(text)
RETURNS text AS
$func$
SELECT public.unaccent('public.unaccent', $1)
$func$ LANGUAGE sql IMMUTABLE;
```

3. **Índice GIN:**
   - Criado sobre a coluna normalizada: `CREATE INDEX idx_foods_name_trgm ON foods USING GIN (f_unaccent(name) gin_trgm_ops);`
4. **Consulta no Modelo (`food.model.ts`):**

```
SELECT * FROM foods
WHERE active = true AND f_unaccent(name) ILIKE f_unaccent($1)
ORDER BY f_unaccent(name) <-> f_unaccent($2) ASC
LIMIT $3 OFFSET $4
```

- O operador `<->` calcula a distância de similaridade trigram, trazendo os resultados mais próximos do termo digitado no topo da lista.

---

6. Agregações e Vigência com `LATERAL JOIN` [CONFIRMADO]

Para a visualização histórica de consumo na página de estatísticas (`GET /api/meals/stats?period=week|month`), a consulta deve correlacionar cada dia consumido com a meta calórica que vigorava **naquela data específica**, sem que alterações posteriores de metas distorçam os dados do passado.

Essa lógica é implementada em `backend/src/models/meal.model.ts` em uma única query SQL de alta performance:

```
WITH daily_consumption AS (
  SELECT
    DATE(m.meal_time) as date_val,
    SUM(mi.calories) as consumed_calories,
    SUM(mi.proteins) as consumed_proteins,
    SUM(mi.carbs) as consumed_carbs,
    SUM(mi.fats) as consumed_fats
  FROM meals m
  JOIN meal_items mi ON mi.meal_id = m.id
  WHERE m.user_id = $1
    AND m.meal_time >= CURRENT_DATE - ($2 || ' days')::interval
  GROUP BY DATE(m.meal_time)
),
daily_goals AS (
  SELECT
    dc.date_val,
    dc.consumed_calories,
    dc.consumed_proteins,
    dc.consumed_carbs,
    dc.consumed_fats,
    ug.calories as goal_calories,
    ug.proteins as goal_proteins,
    ug.carbs as goal_carbs,
    ug.fats as goal_fats
  FROM daily_consumption dc
  LEFT JOIN LATERAL (
    SELECT calories, proteins, carbs, fats
    FROM user_goals
    WHERE user_id = $1 AND effective_from <= dc.date_val
    ORDER BY effective_from DESC
    LIMIT 1
  ) ug ON true
)
SELECT
  TO_CHAR(date_val, 'YYYY-MM-DD') as date,
  consumed_calories,
  consumed_proteins,
  consumed_carbs,
  consumed_fats,
  goal_calories,
  goal_proteins,
  goal_carbs,
  goal_fats
FROM daily_goals
ORDER BY date_val ASC;
```

Vantagens Técnicas do `LATERAL JOIN`:

- **Subconsulta Correlacionada Otimizada:** Para cada linha agrupada da CTE `daily_consumption`, a subconsulta lateral busca unicamente o registro mais recente de `user_goals` com `effective_from <= date_val`.
- **Sem Junções Cartesianas:** Elimina a necessidade de fazer joins completos com filtros no `WHERE` que gerariam produtos cartesianos ou exigiriam funções de janela (`ROW_NUMBER() OVER`) sobre todo o histórico.
- **Zero N+1 Queries:** O backend não precisa iterar em loops na memória para buscar as metas dia a dia; o PostgreSQL resolve toda a projeção analítica em um único plano de execução indexado.

---

6. Convenções de Nomenclatura e Contratos (A-038) [CONFIRMADO]

1. **Banco de Dados e Modelos:** Nomes de tabelas e colunas utilizam estritamente `snake_case` (`meal_time`, `base_unit`, `measured_at`, `effective_from`).
2. **Respostas da API:** Os payloads HTTP retornados pelo backend preservam os nomes do modelo relacional em `snake_case`, evitando camadas arbitrárias de transformação e deserialização.
3. **Contratos Zod Compartilhados (`@caloriesync/shared`):**
    - DTOs de entrada no shared devem respeitar os campos esperados pela camada de serviço e banco.
    - Qualquer discrepância histórica pontual é absorvida nos schemas de validação Zod sem quebrar clientes legados.

---

7. Identificadores UUID e Compatibilidade PostgreSQL (A-065) [CONFIRMADO]

1. **Requisito de Versão:** O CalorieSync requer PostgreSQL 14+ (o ambiente padrão utiliza PostgreSQL 16 via Docker Compose).
2. **Geração Nativa:** A partir do PostgreSQL 13, a função criptográfica `gen_random_uuid()` integra o núcleo do banco de dados (core), não exigindo extensões externas como `pgcrypto`. As migrações novas utilizam `gen_random_uuid()` nativo com garantia de performance e portabilidade. Migrações de baseline mantêm a extensão `uuid-ossp` apenas para compatibilidade histórica de bootstrapping.

---

8. PWA: Manifesto e Service Worker [CONFIRMADO]

Arquivos estáticos em `frontend/public/`, servidos sem passar pelo build do Vite.

Manifesto (`manifest.webmanifest`):

- `id` e `start_url` em `/app`, `scope` em `/`, `display: standalone`, `lang: pt-BR`, categorias `health`, `fitness` e `lifestyle`.
- Ícones: `favicon.svg` (`any`), PNG `icons/icon-192.png` e `icons/icon-512.png` (`any`) e `icons/maskable-512.png` (`maskable`, com margem de segurança para o recorte do Android). O iOS usa `icons/apple-touch-icon.png`, declarado em `index.html`, porque ignora os ícones do manifesto.
- `screenshots/`: três capturas `narrow` (1080×1920: Diário, Histórico, Metas) e uma `wide` (1920×1080: Diário). O Chrome as exibe no diálogo de instalação.
- `shortcuts`: atalhos para Diário (`/app`), Histórico (`/app/historico`) e Metas (`/app/metas`), acessíveis com um toque longo no ícone instalado.

Service Worker (`sw.js`):

- O cache se chama `caloriesync-cache-<CACHE_VERSION>`. No `activate`, caches com outro nome são apagados. **Incremente `CACHE_VERSION` sempre que mudar a lista de pré-cache ou a estratégia**, senão quem já instalou o app continua com os arquivos antigos.
- O pré-cache (`install`) inclui `/spa.html`, o manifesto, `favicon.svg` e os ícones PNG de 192 e 512.
- Navegação usa Network-First (A-101), com fallback para o cache e depois para `/spa.html` quando o usuário está offline. Os demais GETs usam Stale-While-Revalidate.
- Não intercepta `/api/*`, métodos diferentes de GET, `localhost` nem `127.0.0.1`, para evitar conflito com o HMR do Vite.
- Notificações push usam `icons/icon-192.png` como `icon` e `badge`, porque o Android não renderiza SVG em notificação.

Decisões de arquitetura e produto (DECISOES.md)

Registro consolidado das decisões que afetam código, banco e operação. Cada decisão tem identificador estável (`D-xx`) e não deve ser reescrita: mudanças viram emendas explícitas.

> Este arquivo foi consolidado a partir das referências vivas em `docs/`, `README.md` e no código. As decisões abaixo são as citadas pelo repositório até a entrada da D-21.

D-02 — Sessão JWT stateless no cliente

O backend emite JWT HS256 com `userId`. O frontend guarda o token em `localStorage` ("lembrar-me") ou `sessionStorage`. A autorização é feita pelo middleware `authenticate`.

Emenda D-21/A8: o JWT passa a carregar também o claim `tv` (`token_version`). O middleware consulta `users.token_version` por chave primária e rejeita com 401 quando o usuário não existe ou quando `tv` está ausente/divergente. Refresh token e cookie `HttpOnly` continuam previstos para a Fase 8.

D-03 — Ordem de entrega da autenticação

A confirmação de e-mail (BL-12) é pré-requisito da recuperação de senha (BL-05). Não faz sentido permitir reset para contas que ainda não comprovaram posse do e-mail.

Atendimento: a D-21 implementa BL-12 (Fase 3) antes de BL-05 (Fase 4). A recuperação só gera token para contas com `email_verified_at` preenchido.

D-13 — Migrações aplicadas são imutáveis

Nunca editar uma migração já aplicada. Correções e novas estruturas usam arquivos novos com timestamp maior, em ESM (`export const up/down`). Ver `docs/MIGRATIONS.md`.

Atendimento na D-21: as alterações de e-mail/senha entram em migração nova; nenhuma migração existente foi modificada.

D-19 — SSL e proxy reverso

Conexão SSL estrita com o Postgres usa `DB_SSL_CA`; `TRUST_PROXY` aceita número de saltos ou `false` (nunca `true`), para que o rate limiting por IP não seja burlado por header forjado.

D-21 — Confirmação de e-mail (BL-12) e recuperação de senha (BL-05)

Aprovada em bloco pelo prompt de execução autônoma. Vale como especificação vinculante.

A1 — Hard gate no login

Conta sem `email_verified_at` **não recebe JWT**. Não existe "sessão limitada". O login com senha correta e conta não verificada responde **403** com `code: "EMAIL_NOT_VERIFIED"` e dispara um novo código respeitando o cooldown (A5). Senha errada ou usuário inexistente mantém a resposta genérica 401 e o `bcrypt` "dummy" do PC-08.

A2 — Verificação por código

Código de 6 dígitos gerado com `crypto.randomInt`, validade de 15 minutos, máximo de 5 tentativas por código (a 5ª falha invalida o código), uso único, reenvio com cooldown de 60 s e teto de 5 envios por hora por e-mail. Persistência apenas de `HMAC-SHA256(código, AUTH_TOKEN_PEPPER)`; comparação em tempo constante. Um novo código invalida os anteriores.

A3 — Recuperação por link

Token de 32 bytes aleatórios em base64url; persistência apenas do SHA-256; validade de 30 min; uso único. Novo pedido invalida tokens anteriores. Link no formato `${APP_BASE_URL}/redefinir-senha#token=...` (fragmento nunca vai ao servidor). O frontend lê, limpa a URL com `history.replaceState` e envia o token no corpo do POST. `index.html` declara `referrer: no-referrer`.

A4 — Sem enumeração de contas

`register`, `forgot-password` e `resend-code` devolvem sempre a mesma resposta (status, corpo e, na prática, tempo): o e-mail é enviado sem `await` após o commit, com `.catch` que registra o erro sem segredos.

- Cadastro com e-mail **verificado**: não altera nada e envia ao dono o aviso "alguém tentou criar conta com seu e-mail".
- Cadastro com e-mail **não verificado**: sobrescreve nome e hash de senha daquela linha, invalida códigos anteriores e envia novo código (respeitando os tetos).
- `forgot-password`: só gera token para conta existente **e** verificada; caso contrário não envia nada e a resposta é idêntica.

A5 — Login de conta não verificada

Senha correta + conta não verificada: **403** `{ code: "EMAIL_NOT_VERIFIED" }` e novo código (respeitando o cooldown). Senha errada/usuário inexistente: resposta genérica existente, com o `bcrypt` "dummy" do PC-08 para equalizar o tempo.

A6 — Cadastro sem sessão

`POST /api/auth/register` responde **202** genérico, sem JWT. A sessão só é emitida em `POST /api/auth/verify-email` (sucesso) ou no login de conta verificada.

A7 — Reset transacional

Reset bem-sucedido atualiza o hash (bcrypt + política do PC-08), marca o token como usado e incrementa `users.token_version` na **mesma transação** (padrão PC-17). Não faz auto-login. Envia o aviso "sua senha foi alterada" após o commit.

A8 — `token_version` (emenda à D-02)

JWT carrega `tv`. O middleware lê `token_version` do banco por chave primária e rejeita com 401 se o usuário não existir ou o valor divergir; claim ausente é inválido. Troca de senha autenticada no perfil também incrementa `token_version` e devolve token novo. Troca de e-mail autenticada incrementa `token_version`, zera `email_verified_at` e emite novo código de verificação para o endereço novo; a sessão devolvida usa a nova versão.

A9 — Limites

Por IP, `express-rate-limit` (rotas de auth); por e-mail, contagem de linhas nas tabelas de códigos/tokens por janela, sem estado em memória (vale com várias instâncias). Tetos: reset 5/hora por e-mail; `verify-email` 10 tentativas/15 min por IP; `reset-password` 10/hora por IP. A chave do limitador fica isolada em uma única função, pronta para o Turnstile.

A10 — Mailer

Interface `Mailer { send({ to, subject, text, html }) }`, com `SmtpMailer` (nodemailer; Mailpit em dev e SMTP do provedor em produção), `MemoryMailer` (caixa consultável em testes) e escolha por env. Sem SDK de provedor. Proibido logar código, token, URL de reset ou senha. E-mails em pt-BR, multipart, HTML simples, sem imagens remotas e sem rastreamento; o reset informa a validade e "se não foi você, ignore".

A11 — Normalização e cache

E-mail com `trim` + minúsculas antes de qualquer consulta; sem remover `+tag`. Todas as respostas de `/api/auth/*` levam `Cache-Control: no-store`.

A12 — Contas existentes

Contas anteriores à migração ficam **não verificadas** (sem grandfathering). No próximo login com senha correta recebem 403 e um código para confirmar o e-mail.

D-22 — Painel administrativo do catálogo global e ferramentas de curadoria

Aprovada pelo prompt de execução autônoma. Especificação vinculante para RBAC de catálogo, integridade e ferramentas de curadoria.

B1 — Papel sempre do banco

O papel do usuário vem SEMPRE do banco de dados: o middleware de autenticação consulta `role` (e `token_version`) por chave primária a cada requisição. Nunca autoriza por claim do JWT. O papel chega ao frontend pelo payload do usuário (`/auth/login`, `/users/me`), unicamente para controle de interface.

B2 — Criação estrita de administradores

É proibido criar conta admin em migrações, seeds ou variáveis de ambiente. Sem `ADMIN_EMAIL` e sem `ADMIN_DEFAULT_PASSWORD`. O primeiro admin é criado por fluxo natural: cadastro normal, e-mail verificado, e promoção soberana no servidor via CLI: `npm run admin:promote <email>`.

B3 — Escopo estrito ao catálogo global

O administrador gerencia apenas o catálogo global (`foods` com `user_id IS NULL`). Nenhum endpoint admin lê refeições, perfil, metas ou listagem de usuários. Gestão de administradores ocorre exclusivamente via CLI (`promote`, `demote`, `list`).

B4 — Sem exclusão física de alimentos

Alimentos nunca são apagados do banco de dados: apenas ativados ou desativados (`active = true | false`). Não existe rota HTTP de `DELETE` de alimentos.

B5 — Auditoria atômica obrigatória

Toda escrita administrativa (API e CLI) grava na tabela `admin_audit_log` dentro da **mesma transação**: `actor_user_id` (nulo para CLI), `actor_type` (`'user' | 'cli'`), `action`, `entity_type`, `entity_id`, `before` (jsonb), `after` (jsonb), `created_at`. Não existem rotas de edição ou exclusão do log.

B6 — Integridade do histórico

Refeições antigas não são recalculadas nem corrompidas quando um alimento é editado ou desativado. Constatou-se na Fase 1 que a tabela `meal_items` persiste um snapshot imutável dos macronutrientes e calorias. A edição dos campos nutricionais no catálogo é livre para alimentos futuros sem desregular registros retroativos. Consultas de histórico e estatísticas nunca filtram por `active`.

B7 — Barreira contra novos lançamentos com alimentos inativos

Qualquer tentativa de criar item novo de refeição com alimento desativado é rejeitada com status **422** e mensagem contextual explicativa. Na edição de refeição (`PUT /api/meals/:id`), itens pré-existentes

com alimentos inativos são preservados.

B8 — Política de cache do catálogo

O catálogo público possui cabeçalho de cache HTTP `Cache-Control: private, max-age=300` (D-07). A desativação de um alimento pode levar até 5 minutos para sumir das buscas de quem já consultou recentemente. A política é mantida sem alterações.

B9 — Fidelidade nutricional e fontes obrigatórias

É proibido inventar dados nutricionais. Todo alimento oficial novo exige `source` e `source_ref` auditáveis (ex.: "TACO 4ª ed., código X", "rótulo do produto Y", "IBGE medidas referidas"). Sem fonte, a inserção é rejeitada. O agente não escreve listas arbitrariamente: entrega modelo vazio e ferramentas de curadoria.

B10 — Proteção do importador contra sobreposição

A importação automatizada (ex.: `backend/scripts/import-foods.ts`) nunca reativa alimentos desativados nem sobrescreve alimentos curados ou editados manualmente (identificados pelo marcador `admin_edited_at IS NOT NULL`).

B11 — Guarda no roteador administrativo

Todas as rotas sob `/api/admin/*` são protegidas por guarda no roteador raiz: `router.use(authenticate, requireAdmin)`. Requisições não autenticadas recebem 401; usuários comuns autenticados recebem 403 com código `FORBIDDEN`.

B12 — Whitelist estrita de campos e validação por 100 g

Campos permitidos na escrita são rigorosamente filtrados por schemas Zod dedicados. `source`, `user_id` e `active` na criação são controlados pelo servidor. Paginação com limite máximo 100, busca textual com escape de caracteres especiais do ILIKE (`%`, `_`, `\`). Validação dura de valores por 100 g: valores não negativos, kcal ≤ 900, cada macro ≤ 100 g, P+C+G ≤ 100,5 g. Divergência calórica (> 25%) emite alertas (*warnings*) na resposta sem bloquear o salvamento.

D-23 — Paginação server-side e normalização de busca do catálogo administrativo

Aprovada para solucionar a limitação de 20 registros fixos do painel administrativo, garantindo navegabilidade completa sobre milhares de alimentos (TACO, TBCA e rótulos oficiais).

C1 — Coluna gerada armazenada com wrapper unaccent IMMUTABLE

Como a função nativa `unaccent()` do PostgreSQL é `STABLE` (depende do search path / collation), ela não pode ser usada diretamente em colunas geradas ou índices. A migração cria o wrapper seguro `f_unaccent(text) IMMUTABLE` chamando explicitamente `public.unaccent('public.unaccent',

`$1)`. Em vez de índices puramente funcionais, adotou-se a coluna gerada `nome_normalizado VARCHAR(255) GENERATED ALWAYS AS (f_unaccent(lower(name))) STORED`. Essa abordagem:

- Elimina o recálculo de normalização durante leituras e varreduras sequenciais;
- Permite a criação direta de índice GIN (`gin_trgm_ops`) para busca rápida de subsequências (`ILIKE '%termo%'`);
- Permite índice B-Tree composto `(nome_normalizado, id)` para ordenação alfabética rápida e desempate determinístico.

C2 — Separação de consultas COUNT e SELECT

A contagem total de registros filtrados e a busca da página usam queries separadas com a mesma cláusula `WHERE` e parâmetros idênticos. Não foi utilizado `COUNT(*) OVER()` porque este retorna 0 linhas quando a página solicitada for maior que o total de páginas existentes (`page > totalPages`), o que perderia os metadados de paginação e impediria a correção automática pelo cliente.

C3 — Desempate determinístico de ordenação

Toda cláusula `ORDER BY` inclui desempate por chave primária (`id ASC`). Isso garante que os registros permaneçam ordenados de forma estritamente determinística entre páginas, impedindo que alimentos apareçam duplicados ou sejam omitidos durante a navegação paginada.

C4 — Estado na URL como única fonte de verdade

Todos os parâmetros de navegação (`page`, `pageSize`, `q`, `status`, `sort`, `order`) são persistidos na query string via `useSearchParams` do React Router 7. Qualquer mutação de filtros, busca ou ordenação redefine a página para 1. A navegação preserva o contexto no recarregamento da página (F5), histórico de navegação (voltar/avançar) e ao retornar do modal/tela de edição de alimento.

D-24 — Conta de visitante temporária

`POST /api/auth/demo` cria, a cada clique em "Ver demonstração", um usuário `is_guest = true` (e-mail `demo-<uuid>@visitante.invalid`, senha aleatória, e-mail já verificado) com onboarding fixo e 7 dias de refeições. Sem "lembrar de mim".

- **TTL de 2 h:** antes de criar, apaga até 50 visitantes vencidos (refeições primeiro, por causa do `RESTRICT`); o `server.ts` também limpa a cada hora os visitantes vencidos e as linhas vencidas de `rate_limits`. A limpeza filtra só `is_guest = true`. O TTL curto limita a 2 h a indisponibilidade da demo caso alguém lote o teto de propósito.
- **Teto de 500:** com 500 visitantes ativos ou mais, responde 503 `DEMO_UNAVAILABLE`. Contagem e insert rodam na mesma transação sob `pg_advisory_xact_lock`, então cliques simultâneos não passam do teto. A rota tem limite de 20 por IP em 1 h; toda escrita em `/api` tem teto de 300 por IP em 15 min.
- **Ações bloqueadas:** `blockGuestUser` responde 403 `DEMO_READ_ONLY` em `PUT/DELETE /api/users/me`, em `/api/users/me/mfa/{setup,enable,disable}` e em `POST /api/push/subscribe`.

- **Fora de escopo:** a listagem de usuários do admin mostra visitantes misturados.

Manual do Administrador e Curadoria do Catálogo (CalorieSync)

Este documento centraliza as diretrizes, procedimentos e segurança operacional para administração do CalorieSync, conforme a especificação arquitetural **D-22** (decisões B1 a B12).

---

1. Criação e Gestão de Administradores (B1, B2 & B3)

1.1 Princípio de Segurança: Zero Contas Padrão (B2)

- É **estritamente proibido** criar contas de administrador via migrações de banco de dados, sementes (seeds), variáveis de ambiente (`ADMIN_EMAIL`, `ADMIN_PASSWORD`) ou código-fonte.
- O sistema não possui credenciais administrativas pré-configuradas.

1.2 Como Criar o Primeiro Administrador

Para criar o primeiro administrador com total segurança:

1. Acesse o sistema como um usuário normal via tela de cadastro (`/cadastro`) ou registre-se via API.
2. Complete a confirmação de e-mail (fluxo BL-12 de verificação de 6 dígitos enviado por e-mail).
3. No servidor de aplicação, com acesso shell, execute o comando CLI de promoção:

   ```
   npm run admin:promote <email-do-usuario>
   ```

4. O script valida que o usuário existe e possui `email_verified_at` preenchido. Em seguida:
   - Altera `role` para `'admin'` no PostgreSQL;
   - Incrementa `token_version` do usuário para invalidar sessões anteriores;
   - Grava a auditoria em `admin_audit_log` com `actor_type: 'cli'`.
5. No próximo login, o usuário terá acesso imediato às rotas administrativas e verá a aba **"Catálogo"** no menu principal.

1.3 Gestão de Administradores via CLI (B3)

A gestão de papéis administrativos é realizada **exclusivamente via linha de comando** no servidor:

- **Promover usuário:**

  ```
  npm run admin:promote <email>
  ```

- **Rebaixar administrador para usuário comum:**

  ```
  npm run admin:demote <email>
  ```

  Trava de segurança: O script impede o rebaixamento se a operação for deixar o sistema com **zero administradores**.

- **Listar administradores:**

```
npm run admin:list
```

Em ambiente de produção (`NODE_ENV=production`), os comandos de promoção e rebaixamento exigem a confirmação explícita `--yes`.

1.4 Autorização Dinâmica por Requisição (B1)

- O papel (`role`) do usuário é consultado **diretamente no banco de dados por chave primária** (`users.id`) a cada requisição autenticada através do middleware `authenticate`.
- **Nunca** confie em claims do JWT para conceder permissões administrativas.
- O JWT armazena apenas `{ userId, tv }` (versão do token). Quando um admin é rebaixado ou promovido, `token_version` é incrementado e a alteração tem efeito imediato.

---

2. Escopo de Acesso do Administrador (B3)

- O painel administrativo é restrito **exclusivamente à curadoria e gestão do Catálogo Global de Alimentos** (`user_id IS NULL`).
- Nenhum endpoint ou interface administrativa permite visualizar, listar, exportar ou editar dados pessoais de outros usuários, como refeições, medidas corporais, metas ou histórico de navegação.

---

3. Integridade e Histórico de Alimentos (B4, B6, B7)

1. **Sem Deleção Física (B4):**
   - Não existe rota `DELETE` para alimentos no catálogo global.
   - Alimentos indesejados, duplicados ou descontinuados são **desativados** (`active = false`).
2. **Imutabilidade do Histórico de Refeições (B6):**
   - A tabela `meal_items` persiste um **snapshot numérico** completo de calorias e macronutrientes calculados no momento do lançamento.
   - A edição de nutrientes no catálogo global não afeta retroativamente nenhuma refeição lançada no passado.
   - Consultas de histórico diário não filtram por `active = true`, garantindo que refeições passadas continuem sendo exibidas integralmente.
3. **Novas Refeições com Alimentos Inativos (B7):**
   - Tentar adicionar um alimento inativo a uma nova refeição retorna `422 Unprocessable Entity` com feedback claro para o usuário.
   - Em edições de refeições pré-existentes, itens que já continham o alimento inativo são preservados.

---

4. Trilha Imutável de Auditoria (B5)

Todas as ações de escrita administrativa (via API web ou via scripts CLI) gravam uma entrada na tabela `admin_audit_log` **dentra da mesma transação atômica** (`BEGIN / COMMIT`):

- `id`: Identificador único UUID;
- `actor_user_id`: UUID do administrador (quando executado via API) ou `NULL` (quando via CLI);
- `actor_type`: `'user'` (pela web) ou `'cli'` (pelo terminal);
- `action`: Código da ação executada (ex.: `'food_create'`, `'food_update'`, `'food_status_toggle'`, `'admin_promote'`, `'admin_demote'`, `'catalog_curation_apply'`);
- `entity_type`: Entidade afetada (`'food'`, `'user'`);
- `entity_id`: ID do registro afetado;
- `before`: Snapshot em JSONB do estado anterior;
- `after`: Snapshot em JSONB do novo estado;
- `created_at`: Data e hora com fuso horário da gravação.

Não existem rotas para edição ou exclusão do log de auditoria.

---

5. Ferramentas de Curadoria do Catálogo (B9, B10)

O catálogo dispõe de ferramentas automatizadas para revisão e enriquecimento em lote:

1. **Exportar Catálogo para Revisão:**

   ```
   npm run catalog:export
   ```

   Gera `backend/data/catalog-review.csv` (UTF-8 com BOM e separador `;`) contendo heurísticas de duplicatas na coluna `sugestao`.

2. **Aplicar Decisões de Revisão:**

   - **Simulação (Dry-Run padrão):**

     ```
     npm run catalog:apply backend/data/catalog-decisions.csv
     ```

   - **Gravação Efetiva:**

     ```
     npm run catalog:apply -- --apply backend/data/catalog-decisions.csv
     ```

   Marca os registros alterados com `admin_edited_at = NOW()` e grava auditoria.

3. **Importar Alimentos Curados Oficiais:**

   - **Simulação:**

     ```
     npm run catalog:import-curated backend/data/curated-foods.json
     ```

   - **Gravação Efetiva:**

```
npm run catalog:import-curated -- --apply backend/data/curated-foods.json
```

Exige `source` e `source_ref` obrigatórios (B9) e valida limites nutricionais por 100 g.

4. **Proteção da Importação TACO (B10):** A rotina `import-foods.ts` nunca reativa alimentos inativos (`active = foods.active`) e nunca sobrescreve registros que possuam o marcador `admin_edited_at IS NOT NULL`.

---

6. Diretrizes de Segurança em Produção (Nota Arquitetural / Fase 8)

Para ambientes de produção:

1. **Princípio do Menor Privilégio no Banco de Dados:**
   - A aplicação web Express (`DATABASE_URL`) DEVE conectar ao PostgreSQL utilizando um usuário dedicado de banco que possua **apenas privilégios de DML** (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) nas tabelas da aplicação.
   - O usuário da aplicação **NÃO deve ter privilégios de superusuário (superuser)** nem permissões de DDL (`CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`).
2. **Execução de Migrações Isolada:**
   - As migrações (`node-pg-migrate`) devem ser executadas em etapa isolada de pipeline de implantação (CI/CD) utilizando credenciais de banco separadas com permissão de DDL.

---

7. Paginação Server-side e Busca no Catálogo (`/admin/alimentos`)

O painel de catálogo global (`/admin/alimentos` ou `/admin/catalogo`) conta com paginação server-side numerada completa e busca avançada para suportar bases volumosas (milhares de alimentos TACO, TBCA e rótulos oficiais):

7.1 Endpoints Administrativos Paginações

- **Rotas:** `GET /api/admin/foods` (com alias `GET /api/admin/alimentos`).
- **Autenticação:** Protegida pelo middleware raiz `router.use(authenticate, requireAdmin)`.
- **Parâmetros aceitos (Query String):**
  - `page`: Número da página (inteiro $\ge$ 1, default `1`).
  - `pageSize` (ou `limit`): Quantidade de itens por página. Aceita apenas `25`, `50` ou `100` (default `25`); qualquer outro valor aplica o fallback seguro `25`.
  - `q` (ou `search`): Termo de busca por nome ou código (`external_id`/`source_ref`), insensível a maiúsculas e acentos.
  - `status`: Filtro por estado do alimento (`todos`, `ativo`, `inativo` ou sinônimos em inglês `all`, `active`, `inactive`; default `todos`).
  - `sort`: Coluna de ordenação validada por whitelist estrita (`nome` -> `nome_normalizado`, `kcal` -> `calories`, `status` -> `active`). Valores não reconhecidos usam fallback para `nome`.
  - `order`: Direção da ordenação (`asc` ou `desc`, default `asc`).

7.2 Formato da Resposta da API

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Arroz, integral, cozido",
      "calories": 124,
      "proteins": 2.6,
      "carbs": 25.8,
      "fats": 1.0,
      "serving_size": 100,
      "serving_unit": "g",
      "active": true,
      "source": "taco"
    }
  ],
  "meta": {
    "page": 1,
    "pageSize": 25,
    "total": 3412,
    "totalPages": 137
  }
}
```

7.3 Índices e Otimizações de Banco (PostgreSQL)

A migração `1793000000000_add-foods-nome-normalizado-and-pagination-indexes.js` adiciona:

1. **Extensões:** `unaccent` e `pg_trgm`.
2. **Função Imutável:** `f_unaccent(text) IMMUTABLE` como wrapper em torno de `public.unaccent('public.unaccent', $1)`.
3. **Coluna Gerada Armazenada:** `nome_normalizado VARCHAR(255) GENERATED ALWAYS AS (f_unaccent(lower(name))) STORED`.
4. **Índices de Performance:**
    - `idx_foods_nome_normalizado_trgm`: Índice GIN com classe de operador `gin_trgm_ops` para acelerar consultas parciais com `ILIKE '%termo%'`.
    - `idx_foods_nome_normalizado_id`: Índice B-Tree composto para ordenação alfabética rápida com desempate determinístico por `id ASC`.
    - `idx_foods_active`: Índice B-Tree na coluna `active` para aceleração de filtros por status.

7.4 Persistência de Estado na URL e Experiência do Usuário (Frontend)

- **Single Source of Truth:** O estado da tabela (`page`, `pageSize`, `q`, `status`, `sort`, `order`) reside na query string da URL via `useSearchParams` do React Router 7.
- **Navegação Sem Perda:** F5, links compartilhados, botões voltar/avançar e retorno após edição de item preservam fielmente a página e os filtros exatos do operador.
- **Debounce:** O campo de busca conta com debounce automático de 300 ms, além de acionamento imediato por tecla `Enter` ou clique em "Buscar".
- **Resets Inteligentes:** Alterar o termo de busca, filtro de status, coluna de ordenação ou tamanho de página redefine automaticamente a página para `1`.

- **TanStack Query & Prefetching:** Utiliza `placeholderData: keepPreviousData` (evitando telas piscando) e realiza prefetch automático em segundo plano da próxima página.
- **Acessibilidade & Mobile:** Componente `Pagination` com marcação semântica `<nav aria-label="Paginação">`, atributos `aria-current="page"`, janelamento com reticências no desktop e modo compacto para visualização em telas móveis.

Guia e Estratégia de Testes (TESTING.md)

Este documento estabelece as diretrizes, comandos, suítes existentes, convenções e o mapeamento do que **não** está coberto pela suíte de testes do **CalorieSync**.

---

1. Como Rodar [CONFIRMADO]

Os comandos oficiais atuais são `npm test`, `npm run test:strict`, `npm run test:file -- tests/integration/meal_items_batch.test.ts` e `npm run test:frontend` (ou `npm --prefix frontend test`). Os aliases `test:backend:file` e `test:frontend:file` permanecem disponíveis. Não invoque `tsx --test`, `npx` ou Vitest avulsos. Todos os comandos de teste usam o wrapper de processos com teto global de 300 s, ajustável por `TEST_PROCESS_TIMEOUT_MS` (1..3600000 ms). Comandos longos adicionais usam `node scripts/bounded-command.mjs <comando> <argumentos>`.

O runner resolve os caminhos a partir do script, fixa o CWD, `NODE_ENV=test`, `DB_NAME=contador_calorias_test` e o fuso. O `.env` vem exclusivamente do caminho absoluto do backend; variáveis já definidas no processo prevalecem. Variáveis obrigatórias vazias/ausentes falham via Zod sem imprimir seus valores. Não altere o `.env` para testes negativos: passe a variável vazia no ambiente do processo filho.

Toda suíte que alcance pool/app, inclusive por services, models ou helpers, registra `closeTestPool()` de `backend/tests/helpers/lifecycle.ts` antes dos testes; passe a limpeza como callback. O helper executa a limpeza em `try` e `await pool.end()` em `finally`. Não combine encerramentos próprios com o helper. A guarda AST `node scripts/check-test-hygiene.mjs`, integrada ao lint, verifica imports transitivos locais.

`test` usa `--test-force-exit` como proteção extra. `test:strict` e `test:file` não usam essa flag e desabilitam `allowExitOnIdle` para expor sockets esquecidos. Execute strict na verificação final e no CI. O timeout de 30 s do Node é herdado pelos subtestes e não substitui o teto global: handles após o fim dos testes ainda podem manter o processo vivo. Frontend: `vitest run`, teste/hook 20 s e teardown 10 s (`frontend/vite.config.ts`); MSW fecha em `afterAll`.

Os resultados na tabela abaixo representam o status verificado da suíte de testes.

Comandos executados a partir da **raiz** do monorepo:

```
# 1. Testes de Backend (Node Native Test Runner contra banco isolado)
npm run test

# 2. Testes de Frontend (Vitest + RTL + MSW)
npm run test:frontend

# 3. Validação de Linter (ESLint em todo o projeto)
npm run lint

# 4. Validação de Build e Tipagem (TypeScript + Vite)
npm run build
```

Simular o CI antes do push

```
npm run test:full
```

Roda os mesmos passos de `.github/workflows/ci.yml`, na mesma ordem, e para no primeiro que falhar: audit, gitleaks, typecheck, format, knip, lint, testes do backend com cobertura, rollback de migrações, carga, E2E, testes do frontend com cobertura, build e `docker build`. Leva de 3 a 5 minutos.

Pré-requisitos: `npm run docker:up` (PostgreSQL) e `backend/.env` com `DATABASE_URL_TEST`. O gitleaks (`winget install Gitleaks.Gitleaks`) e o Docker Desktop são opcionais: sem eles, os passos são pulados com aviso no final. O teste de carga usa a porta `3201`, então pode rodar com o `dev:all` aberto.

Diferenças em relação ao CI:

- **Variáveis de ambiente:** localmente vêm do `backend/.env`; no CI, do bloco `env:` de cada passo do `ci.yml`. Ao criar um script que dependa de uma variável nova, adicione-a também ao `ci.yml`, senão o passo passa aqui e falha lá.
- **Arquivos gerados no CI:** a action do gitleaks grava `results.sarif` na raiz (ignorado no `.prettierignore`); o gitleaks local não gera esse arquivo.
- **Manutenção:** o script (`scripts/run-full-verification.mjs`) é uma cópia da lista de passos do `ci.yml`. Ao adicionar ou mudar um passo no CI, atualize o script também.

---

2. O Que Existe e Status Verificado em Sessão [CONFIRMADO]

Abaixo está o status real verificado através da execução direta dos comandos no ambiente local:

Camada	Ferramenta / Runner	Total de Testes	Status Atual	Detalhes / Observações
Backend	`node:test` + `node:assert` + `supertest` via `tsx --test`	**263 testes em 52 suítes**	✅ **263 PASSOU / 0 FALHOU** (~41s)	Roda exclusivamente contra o banco `contador_calorias_test` com trava `_test` e limpeza via `cleanupTestData`.
Frontend	`Vitest 5.0.3` + `React Testing Library` + `MSW 2.15.0` + `@vitest/coverage-v8`	**185 testes em 26 arquivos**	✅ **185 PASSOU / 0 FALHOU** (~19s)	Suporte completo a relatório de cobertura v8 via `npm --prefix frontend run test:coverage`.
Linter	`ESLint 10` + `check-test-hygiene.mjs` + `check:colors`	Monorepo completo	✅ **0 Erros / 1 Warning**	Higiene de testes, guarda AST de pool, guarda de cores semânticas e regras de código sem apontamentos.
Build	`tsc -b && vite build`	Frontend	✅ **Compilação com Sucesso**	Code-splitting por rota com chunks separados de vendor (`react-vendor`, `tanstack-vendor`, `chart-vendor`, etc.).

Suítes Existentes no Backend (`backend/tests/`) [CONFIRMADO]

1. **Unitários (`tests/unit/`):**
   - `architecture.test.ts`: Regras arquiteturais de 4 camadas (routes -> controllers -> services -> models) e ausência de ciclos com ArchUnitTS.
   - `nutrition.service.test.ts`: Validação matemática de cálculo de TMB (Mifflin-St Jeor, Katch-McArdle com LBM), TDEE, superávit/déficit e distribuição de macros para IMC alto conforme D-15.
   - `meal.test.ts`: Cálculos nutricionais proporcionais de itens (`calculateItemNutrients`) para porções em gramas, ml e unidades.
   - `auth.middleware.test.ts`: Claim `tv` obrigatório, divergência de `token_version`, usuário removido e cabeçalhos ausentes (BL-12 / A8).
   - `authTokens.test.ts`: Geração de código, HMAC-SHA256 com pepper, token base64url, SHA-256, comparação em tempo constante e normalização de e-mail (BL-12/BL-05).
   - `mailer.test.ts`: `MemoryMailer`, templates pt-BR e ausência de segredos nos conteúdos (D-21/A10).
   - `date.util.test.ts`: Conformidade de datas civis e fuso horário `America/Sao_Paulo` (D-01).
   - `db_cleanup.test.ts`: Trava de segurança de runtime checando `_test` e padrão `tester-%` (D-11 / PC-14).
   - `input.limits.test.ts`: Validação de tetos de payload e sanitização Zod (CA-11).
   - `password_security.test.ts`: Política de senha forte e mitigação de timing attacks via compare dummy (PC-08).
   - `pool.ssl.test.ts`: Validação de SSL estrito em produção com suporte a certificado CA customizado (CA-05 / D-19).
   - `reset_db_guard.test.ts`: Trava de segurança impedindo execução de reset de banco fora do ambiente isolado (CA-12).
   - `routes_registration.test.ts`: Verificação de unicidade no registro de sub-roteadores no Express (PC-04).
   - `validate.middleware.test.ts`: Validação particionada de body, query e params (CA-15).
   - `threadpool.test.ts`: Dimensionamento do pool de threads libuv (BL-17 / PERF-011).
   - `trust_proxy.test.ts`: Configuração e validação de saltos de proxy reverso (BL-01 / D-19).
2. **Integração (`tests/integration/`):**
   - `user.routes.test.ts`: Registro, login, obtenção de perfil, fluxo de onboarding, prévia e atualização de metas com vigência, atualização protegida de perfil com senha e exclusão com senha.
   - `meal.routes.test.ts`: Criação de refeição com múltiplos itens, listagem de refeições do dia (`/today`), atualização com substituição atômica, exclusão e estatísticas por período.
   - `food.routes.test.ts`: Busca de alimentos com tolerância a acentos (`unaccent`) e caixa (`ILIKE`), proximidade trigram (`pg_trgm`), paginação, cache privado HTTP (PC-03 / D-07) e projeção explícita de colunas com `source` (PC-09 / D-08).
   - `authorization.test.ts`: Testes rigorosos de autorização multiusuário (IDOR) cobrindo isolamento total de refeições, metas e dados de perfil (PC-18 / D-14).

- `user_transactions.test.ts`: Transações atômicas com `BEGIN`/`COMMIT`/`ROLLBACK` e rollback automático sob falhas em onboarding, perfil e metas (PC-17).
- `health.routes.test.ts`: Probe ativo de saúde do banco de dados no contrato `{ success, data }` com timeout dedicado (PC-11 / A-040).
- `meal_items_batch.test.ts`: Inserção multi-row em lote de itens de refeição (PC-22 / D-14).
- `auth_rate_limit.test.ts`: Rate limiting de autenticação por IP e por e-mail com limite global (BL-01 / SEC-006 / A-117).
- `auth_email_verification.test.ts`: Cadastro 202 sem sessão, hard gate 403 `EMAIL_NOT_VERIFIED`, código expirado/5 tentativas/uso único, reenvio com cooldown e teto, conta verificada e troca de e-mail (BL-12 / A1–A6).
- `auth_password_reset.test.ts`: Resposta genérica, token de uso único/expirado, política de senha, incremento de `token_version` e ausência de auto-login (BL-05 / A3/A4/A7/A8).
- `foods_uniqueness.test.ts`: Índices parciais de unicidade multi-tenant em catálogo e personalizados (A-003 / BL-09 / A-118).
- `user_measurements.test.ts`: Evolução de medidas corporais, ordenação cronológica e não-sobrescrita retroativa (BL-21 / FEAT-C04 / A-119).
- `request_logging.test.ts`: Logging estruturado assíncrono com correlation ID e sincronização determinística (BL-08 / ARCH-020 / A-120).
- `saved_meals.test.ts`: Gerenciamento de refeições salvas e combos com cálculo nutricional.
- `push_notifications.test.ts`: Gerenciamento de inscrições de push notifications (BL-27).
- `admin_rbac.test.ts`: Role padrão 'user', CHECK constraint, autenticação recarregando role do banco a cada requisição (B1), e rebaixamento imediato sem esperar expiração de token.
- `admin_foods.test.ts`: Matriz de autorização com guard em `/api/admin` (401/403/200), CRUD administrativo, auditoria atômica (B5), rejeição de item inativo novo na refeição com 422 (B7), validação de 100g e warnings de divergência calórica.
- `admin_cli.test.ts`: CLI de administração de usuários (`admin:promote`, `admin:demote`, `admin:list`), validação de e-mail verificado, proteção contra remoção do último admin, auditoria CLI (`actor_type: 'cli'`), incremento de `token_version`.
- `catalog_curation.test.ts`: Curadoria de catálogo: exportação CSV (BOM, `;`), aplicação de decisões de curadoria (simulação por padrão, `--apply`), importação de alimentos curados com `source` e `source_ref` obrigatórios, e proteção do script `import-foods.ts` contra reativação e sobrescrita de curados (B10).

3. **Carga (`tests/load/autocannon.js`):**
   - Script de estresse com `autocannon` para medição de requisições por segundo e latência da API sob concorrência.

Suítes Existentes no Frontend (`frontend/src/`) [CONFIRMADO]

1. `src/tests/architecture.test.ts`: Regras arquiteturais do frontend (isolamento de services sem importar componentes UI, componentes desacoplados de páginas e ausência de ciclos) com ArchUnitTS.

2. `src/tests/router.test.tsx`: Testes de roteamento SPA declarativo via `react-router` v7, redirecionamentos e histórico.
3. `src/tests/navigation.test.tsx`: Navegação profunda por finalidade, roteamento de abas, rotas protegidas e breadcrumbs.
4. `src/tests/api.test.ts`: Resiliência contra respostas não-JSON e disparo de evento `auth:unauthorized` sob 401.
5. `src/tests/auth_storage.test.tsx`: Sanitização do `localStorage`, privacidade de biometrias e hidratação segura (A-129).
6. `src/tests/LoginPage.test.tsx`: Renderização da tela de login e alternância de abas.
7. `src/components/auth/LoginForm.test.tsx`: Renderização, preenchimento, submissão, erro 401 genérico, encaminhamento 403 `EMAIL_NOT_VERIFIED` e link de recuperação.
8. `src/components/auth/RegisterForm.test.tsx`: Validação de senha forte, termos LGPD, resposta 202 genérica e ausência de sessão antes da verificação.
9. `src/components/meals/NewMealModal.test.tsx`: Autocomplete, paginação, combos salvos, edição inline e alimentos customizados.
10. `src/pages/private/__tests__/DiaryPage.test.tsx`: Resumo de calorias, expansão de macros, estados vazios e registro de refeições.
11. `src/pages/private/__tests__/StatsPage.test.tsx`: Renderização de métricas e gráficos nutricionais via Recharts.
12. `src/pages/private/__tests__/SettingsSeparation.test.tsx`: Isolamento das tarefas de perfil, rotina, segurança e privacidade.
13. `src/pages/public/__tests__/LegalPage.test.tsx`: Páginas públicas de Termos de Uso, Política de Privacidade e Segurança.
14. `src/styles/tokens.contrast.test.ts`: Validação WCAG AA de contraste para todos os tokens de cores no modo claro e escuro.
15. `src/pages/public/VerifyEmailPage.test.tsx`: Estado do router sem persistência de e-mail, campo `autocomplete="one-time-code"`, contagem de 60 s, código inválido, verificação com sessão e reenvio (BL-12).
16. `src/pages/public/ForgotPasswordPage.test.tsx`: Validação local de e-mail e resposta genérica idêntica para conta existente ou inexistente (BL-05 / A4).
17. `src/pages/public/ResetPasswordPage.test.tsx`: Leitura do token no fragmento, limpeza da URL, senha forte/confirmação, sucesso sem sessão (e encerramento da sessão local antiga) e token inválido/expirado (BL-05 / A3).
18. `src/pages/admin/__tests__/AdminCatalogPage.test.tsx`: Painel de administração de catálogo, proteção de rota por `AdminRoute`, tabela desktop e cards mobile, badges de status, modais com avisos de divergência calórica e diálogo de confirmação.

---

3. Convenções e Regras de Testes [CONFIRMADO]

Backend: Isolamento Mandatório de Banco

- **Fail-Safe no Pool (`backend/src/database/pool.ts`):** O pool de conexões possui uma trava em código que verifica:

```
if (currentEnv === 'test' && !dbName.endsWith('_test')) {
  console.error('FATAL: Testes devem rodar exclusivamente em banco terminado em _test!');
  process.exit(1);
}
```

Se qualquer desenvolvedor ou agente tentar rodar a suíte contra `contador_calorias` de desenvolvimento, o runner aborta instantaneamente.

- **Limpeza Determinística Entre Testes (D-11 / PC-14):**
  - Conforme decisão D-11, vigora uma exceção explícita e restrita à proibição de deleções em massa do `AGENTS.md` §4:
  - É permitido exclusivamente no código de testes (hook `after`):

    ```
    DELETE FROM users WHERE email LIKE 'tester-%';
    ```

    (O `ON DELETE CASCADE` do esquema cuida da remoção das tabelas filhas `meals`, `meal_items`, `user_goals`, `body_measurements`).
  - **Trava de Runtime Obrigatória:** O teste deve checar ativamente se `current_database()` termina com `_test` antes de deletar (abortar caso contrário).
  - É terminantemente proibido o uso de `TRUNCATE` ou `DROP TABLE/SCHEMA`.

Frontend: Acessibilidade e MSW

- **Seleção por Acessibilidade (RTL):** Elementos devem ser buscados via `getByRole`, `getByLabelText`, `getByPlaceholderText`, simulando como o usuário interage. **É terminantemente proibido** buscar elementos por classes CSS (`.btn`, `.bg-primary`) ou seletores hierárquicos genéricos (`div > span`).
- **Interceptação de Rede (MSW):** Todas as chamadas de API são interceptadas em `src/mocks/handlers.ts`. Não são permitidos mocks manuais de `global.fetch` em arquivos de teste individuais.

---

4. O Que NÃO Está Coberto (Gaps Reais da Suíte) [CONFIRMADO]

Os seguintes itens representam oportunidades futuras de ampliação de testes:

Ponta a Ponta (E2E):

1. **Testes E2E Completos (Playwright / Cypress):** Inexistentes no repositório. O projeto prioriza testes de integração rápida no backend e testes de componentes com MSW no frontend.
2. **Concorrência Extrema de Conexões:** Cenários de centenas de usuários concorrentes salvando refeições na mesma fração de segundo são cobertos apenas por scripts manuais de estresse (`autocannon.js`).

Migrações de banco (`backend/migrations/`)

Migrações versionadas gerenciadas por `node-pg-migrate`. Cada arquivo é aplicado uma única vez e registrado na tabela `pgmigrations`.

Regras

1. **Migrações aplicadas são imutáveis (D-13).** Nunca edite um arquivo já aplicado. Toda correção de esquema deve ser feita por uma **nova** migração versionada com prefixo de timestamp maior que o da última existente.
2. **Novas migrações devem usar ESM nativo.** `backend/package.json` declara `"type": "module"`, portanto o formato obrigatório é:

   ```
   export const shorthands = undefined;
   export const up = (pgm) => { /* ... */ };
   export const down = (pgm) => { /* ... */ };
   ```

3. **Sempre implemente `down` reversível** (ou documente explicitamente por que a reversão é parcial/ambígua, como em dados já deduplicados).
4. **Nunca use comandos destrutivos** (`DROP TABLE`, `DROP DATABASE`, `TRUNCATE`) sem autorização prévia do usuário.
5. **O banco de teste é migrado automaticamente.** `scripts/run-tests.mjs` roda `node-pg-migrate up` em `DATABASE_URL_TEST` antes de cada execução do backend; nada a fazer à mão.

Migrações legadas congeladas (BL-15 / ARCH-017 / D-13)

As migrações abaixo foram escritas no formato **CommonJS** (`exports.up = ...`) antes da adoção do ESM e ficam **congeladas** por D-13 (imutabilidade pós-aplicação):

- `1791310642375_history-tables.js`
- `1791390511650_add-food-scale-features.js`

Elas permanecem funcionais no Node atual (v24) porque o detector de sintaxe de módulo do Node carrega arquivos `.js` sem sintaxe ESM como CommonJS automaticamente (comportamento estável a partir do Node 20.19 / 22.7, versões mínimas suportadas).

Não converta esses arquivos para ESM — isso violaria D-13. A padronização vale apenas para as migrações novas. Caso uma futura migração precise alterar algo criado por uma delas, crie uma nova migração.

Migração de Paginação e Busca do Catálogo (D-23)

- **Arquivo:** `1793000000000_add-foods-nome-normalizado-and-pagination-indexes.js`
- **Formato:** ESM nativo (`export const up/down`).
- **Objetivo:** Adicionar suporte a buscas e ordenações eficientes sem distinção de acento e caixa no catálogo global de alimentos (`foods`).
- **Recursos Criados:**
  - Extensões PostgreSQL `unaccent` e `pg_trgm`.
  - Função `f_unaccent(text) IMMUTABLE` em schema public.
  - Coluna gerada `nome_normalizado VARCHAR(255) GENERATED ALWAYS AS (f_unaccent(lower(name))) STORED`.
  - Índice GIN `idx_foods_nome_normalizado_trgm` com operador `gin_trgm_ops` para aceleração de `ILIKE '%termo%'`.
  - Índice B-Tree composto `idx_foods_nome_normalizado_id` em `(nome_normalizado, id)` para paginação determinística.
  - Índice B-Tree `idx_foods_active` na coluna `active`.
- **Reversibilidade (`down`):** Remove os índices, a coluna gerada, a função wrapper e as extensões em ordem estritamente reversa.

Papéis de banco e RLS (issues #9 e #10)

Papéis (#9)

Papel	Uso	Privilégios
`app_user`	API (`DB_USER`/`DB_PASSWORD`), CLIs `admin:*` e `catalog:*`	`SELECT/INSERT/UPDATE/DELETE` e uso de sequences. Sem DDL. Sem `UPDATE/DELETE` em `admin_audit_log`.
`migration_user`	`npm run setup` (`DATABASE_URL`), `backup-db.mjs`	Dono dos objetos: DDL, `CREATE EXTENSION`. Nunca usado pela API.

A separação já existia no código: a API lê `DB_*` e as migrations leem `DATABASE_URL`. Faltava só criar os papéis.

Provisionar (uma vez por banco, como superusuário ou dono do banco)

```
psql "$ADMIN_DATABASE_URL" -v ON_ERROR_STOP=1 \
  -v app_password="'<senha_app>'" -v migration_password="'<senha_migracao>'" \
  -v dbname=contador_calorias -f backend/scripts/provision-db-roles.sql
```

Depois: `DB_USER=app_user` (+ senha) para a API e `DATABASE_URL=postgresql://migration_user:...` para as migrations. Rode o script também depois da **primeira** migration se o banco já tinha tabelas criadas por outro dono (o script cobre as existentes e, via `ALTER DEFAULT PRIVILEGES`, as futuras criadas por `migration_user`).

Observações:

- A senha é definida num `ALTER ROLE ... PASSWORD` separado. `NOSUPERUSER NOCREATEDB NOCREATEROLE` só é aplicado (via `\gexec`) se o papel tiver algum desses atributos: no Neon/Postgres 16 o dono do banco não é superusuário, e tocar em `SUPERUSER` falha com "Only roles with the SUPERUSER attribute may change the SUPERUSER attribute".
- `ALTER DEFAULT PRIVILEGES FOR ROLE migration_user` e `ALTER ... OWNER TO migration_user` exigem que o executor herde e possa assumir `migration_user`. Sem superusuário (Neon), o script faz `GRANT migration_user TO <executor>` (ex.: `GRANT migration_user TO neondb_owner;`) antes do `ALTER DEFAULT PRIVILEGES`, só quando falta. No PG16 quem cria o papel já é membro com `ADMIN OPTION`, mas sem `INHERIT`/`SET`, e isso não basta.
- Tabelas já criadas pelo dono (`postgres` local ou `neondb_owner` no Neon) continuam dele. Para que `migration_user` as altere, transfira a posse depois do script (decisão do operador, não feita no script). **Não use `REASSIGN OWNED`**: no Neon ele falha porque tenta transferir também o próprio banco. Use:

```
DO $$
DECLARE r record;
BEGIN
  FOR r IN SELECT tablename FROM pg_tables
           WHERE schemaname = 'public' AND tableowner <> 'migration_user' LOOP
    EXECUTE format('ALTER TABLE public.%I OWNER TO migration_user', r.tablename);
  END LOOP;
  -- Sequences de serial/identity mudam junto com a tabela; aqui ficam as avulsas.
  FOR r IN SELECT sequencename FROM pg_sequences
           WHERE schemaname = 'public' AND sequenceowner <> 'migration_user' LOOP
    EXECUTE format('ALTER SEQUENCE public.%I OWNER TO migration_user', r.sequencename);
  END LOOP;
END $$;
ALTER FUNCTION public.f_unaccent(text) OWNER TO migration_user;
```

- `CREATE EXTENSION` de `unaccent`/`pg_trgm`/`uuid-ossp` é permitido a não superusuário por serem *trusted* (PG13+), com `CREATE` no banco.
- Em dev e nos testes segue valendo o usuário `postgres` do docker-compose; o script é para staging/produção.

RLS (#10): avaliação

Recomendação: não adotar agora.

- Os models usam `pool.query` direto. RLS com `SET LOCAL app.user_id` exige transação por requisição (ou `set_config` por conexão) em todo acesso: reescrita ampla da camada de models e uma conexão presa por request.
- Falha de configuração é silenciosa e perigosa: sem `app.user_id` a policy devolve vazio ou, com o papel errado (dono da tabela, `BYPASSRLS`, superusuário), ignora a policy.
- Admin, CLIs e catálogo global (alimentos sem `user_id`) precisam de policies/bypass próprios.
- O isolamento por usuário já é coberto por `WHERE user_id = $n` e pela suíte `authorization.test.ts` (IDOR/BOLA).

Reavaliar se surgir acesso ao banco fora da API (BI, suporte) ou mais de uma aplicação no mesmo schema. Nesse caso: começar pelas tabelas de saúde (`meals`, `body_measurements`, `user_goals`), `FORCE ROW LEVEL SECURITY`, `app_user` sem `BYPASSRLS`.

Organização das telas do CalorieSync

Decisão de 19/10/2026: organizar a navegação pelas tarefas do usuário, preservando o Diário (antigo Dashboard) e as APIs existentes.

Referências e critérios

- NN/g — Information Architecture vs. Navigation: definir agrupamentos e rótulos coerentes antes de escolher os controles de navegação.
- NN/g — Progressive Disclosure: manter tarefas frequentes visíveis e distribuir opções especializadas em páginas secundárias com acesso claro.

Aplicação ao projeto: navegação principal persistente com duas áreas, mais a área da conta; navegação local com URLs próprias; formulários independentes por finalidade. É uma decisão de arquitetura baseada nesses princípios e nas tarefas existentes, a validar com uso real; não existe uma divisão universal para toda SPA.

Mapa implementado

O app autenticado vive sob `/app` (bloqueado no `robots.txt`, `noindex` via `useNoIndex`). `/` é a landing pública; `/`, `/termos`, `/privacidade` e `/seguranca` são pré-renderadas no build (`frontend/scripts/prerender.mjs`, metadados em `frontend/src/seo/pages.ts`). URLs antigas sem o prefixo (`/historico/*`, `/metas/*`, `/configuracoes/*`, `/conta`, `/perfil`, `/diario/:date`, `/onboarding`) redirecionam para `/app/...`.

Área	Página	Responsabilidade
Diário	`/app`	Visão e registro do dia; conteúdo preservado
Histórico	`/app/historico/alimentacao`	Média e histórico do consumo alimentar
Histórico	`/app/historico/corpo`	Histórico de peso/gordura e nova medição
Metas	`/app/metas/rotina`	Objetivo e nível de atividade física
Metas	`/app/metas/diarias`	Metas automáticas ou manuais de calorias e macros
Perfil	`/app/perfil`	Nome, e-mail (não alterável), altura, sexo e nascimento
Configurações	`/app/configuracoes/notificacoes`	Ativação de lembretes e envio de teste
Configurações	`/app/configuracoes/seguranca`	Alteração de senha
Configurações	`/app/configuracoes/privacidade`	Exportação dos dados e exclusão da conta

Estatísticas e histórico corporal passam a compartilhar a área Histórico, com páginas separadas para preservar os contextos e filtros próprios. Peso/gordura têm um formulário de registro com seletor de data padronizado; altura é alterada diretamente no perfil. Objetivo e rotina ficam juntos porque orientam o cálculo automático; metas numéricas têm confirmação própria.

Cada formulário envia apenas os campos da tarefa atual. O e-mail cadastrado é exibido no perfil mas não é alterável; alterar senha é uma tarefa independente na tela de Segurança. Em Metas diárias, AUTOMÁTICO é só visualização (valores calculados, TMB e gasto total) e nunca grava sozinho; MANUAL só grava ao confirmar em Aplicar metas. Se a meta salva é manual e o usuário escolhe AUTOMÁTICO, a troca só vale depois de confirmar em Usar metas automáticas; trocar de modo sem confirmar descarta o rascunho e volta ao salvo. Atualizações invalidam as consultas relacionadas para manter o Diário e as metas sincronizados.

Navegação e compatibilidade

- Header do desktop com Diário e Histórico e indicação da área atual; o menu da conta espelha a tela `/app/conta`: Metas, Perfil, Configurações e Catálogo (admin).
- Links locais preservam acesso direto, recarga e histórico de voltar/avançar. As páginas permanecem protegidas pela autenticação e pelo onboarding.
- `/estatisticas` e `/evolucao` (com subrotas) redirecionam para `/app/historico/alimentacao`; `/medidas` e `/evolucao/corpo` para `/app/historico/corpo`.
- `/app/historico`, `/app/metas` e `/app/configuracoes` abrem suas primeiras páginas por redirecionamento com substituição no histórico.
- Nenhuma mudança de banco ou migration é necessária.

Navegação mobile, tela Conta e swipe

- **Barra inferior (abaixo de `md`, 4 slots):** `[DIÁRIO] [HISTÓRICO] [+] [CONTA]`. Vidro escuro translúcido sem divisórias; só ícones, sem legendas (o nome acessível vem do `aria-label`); o ícone ativo é preenchido (`aria-current="page"`). O "+" abre o modal de Nova Refeição em qualquer tela (data em exibição no Diário; hoje nas demais) e nunca fica ativo.
- **Auto-ocultar:** some ao rolar para baixo e volta ao rolar para cima; fica sempre visível no topo (`scrollY < 56`), com foco por teclado dentro dela e em `/app/configuracoes/*`. Some enquanto um campo de texto/select está em foco, para não subir com o teclado.
- **HISTÓRICO** abre `/app/historico` e fica ativo só nessa seção. Metas é uma tela separada, acessada pela Conta.
- **`/app/conta`:** hub com cartão do usuário, atalhos para Metas, Perfil e Configurações (e Catálogo, para administradores), alternador de tema e "Sair da conta". Ativa em `/app/conta`, `/app/metas/*`, `/app/perfil` e `/app/configuracoes/*`. No mobile, o menu do avatar do header é substituído por esta tela; no desktop, o menu da conta oferece os mesmos atalhos.
- **Abas de seção:** Histórico, Metas e Configurações têm um único nível de abas, com `SegmentedNav` (caixa e `bg-primary` no ativo). Colunas iguais, sem rolagem horizontal, rótulo curto quando o nome é longo.
- **Filtros ficam no card que afetam:** período (7D | 30D) em Alimentação e métrica (PESO | % GORDURA) em Peso e medidas são `SegmentedToggle` (chips) no cabeçalho do `SectionCard`; o filtro permanece visível nos estados de carregando, erro e vazio. O nome da aba não é repetido na página (título `sr-only` só para a hierarquia de cabeçalhos).
- **Resumos sem grade 2x2:** Alimentação mostra consumo × meta e P/C/G num único card; Peso e medidas mostra Atual/Menor/Maior/Variação numa faixa de 4 colunas dentro do card do

gráfico.
- **Swipe horizontal entre os destinos da barra inferior** (Diário ↔ Histórico ↔ Conta), só no mobile e só em toque, sem dar a volta. A tela acompanha o dedo; ao soltar, sai deslizando e a nova entra pelo lado oposto (ou volta ao lugar, com resistência na ponta da fila). Confirma por distância (o maior entre 64 px e 30% da largura) ou por flick (≥ 40 px a ≥ 0,5 px/ms). Com `prefers-reduced-motion`, navega direto. **Não há swipe entre abas irmãs** (Alimentação ↔ Peso e medidas, abas de Configurações): elas só mudam pela barra segmentada. Telas fora da barra (ex.: `/admin/alimentos`, páginas legais) não têm swipe. Como o Diário agora usa o gesto horizontal, a futura navegação por dia (BL-02) precisará de outro gesto. Detalhes em `docs/SPEC.md` (RF-008).
- **Fonte única:** todos os destinos vêm de `frontend/src/navigation/nav-items.ts`.

Verificação

Teste inicial reproduz a presença indevida de medidas corporais no perfil. Regressões cobrem separação das telas, payloads independentes, senha para alteração do e-mail, metas automáticas/manuais, rotas protegidas, links antigos e navegação pelo histórico.

Guia de Cores e Identidade Visual — CalorieSync

Este documento documenta de forma completa todas as cores, variáveis, códigos hexadecimais e regras de sombra do projeto **CalorieSync** (extraídos do Diário, `tailwind.config.js`, `index.css` e da identidade brutalista da aplicação).

> **Revisão v2 — "Neo-Brutalismo de Tinta":** o modo claro foi redesenhado porque o verde neon `#CCFF00` tinha contraste de apenas **1,17 : 1** sobre o branco (praticamente invisível). O modo escuro permanece como estava.
>
> **Revisão v3 — "Ajuste de Acessibilidade Destructive":** no modo claro, `--destructive` foi ajustado para `#B91C1C` (`hsl(0, 74%, 42%)`) porque `#EF4343` dava apenas ~3,8:1 sobre branco (reprovando WCAG AA para texto normal). No modo escuro permanece `#EF4343` (`hsl(0, 84%, 60%)`).

🧭 Regra de Ouro do Verde Neon

> **O `#CCFF00` é uma cor de PREENCHIMENTO (fill), não de TINTA.**
>
> - **Fill (`bg-primary`):** botões, barras de progresso, badges, marca-texto do logo — sempre com **texto preto** e, no modo claro, **borda preta de 2px**. Idêntico nos dois temas.
> - **Tinta (`text-primary-ink`):** textos e ícones verdes sobre o fundo. No modo escuro é o próprio `#CCFF00`; no modo claro vira o verde-oliva `#4D6600`.
> - **Nunca** use `text-primary` para texto/ícone sobre fundo claro.

⚡ Resumo Direto: A sombra dos cards é outra cor ou é a mesma com transparência?

> **Depende do tema (agora controlado por variáveis CSS):**
>
> - **Modo Escuro:** EXATAMENTE o verde neon `#CCFF00` com transparência — `rgba(204, 255, 0, 0.2)` normal e `rgba(204, 255, 0, 0.4)` no hover.
> - **Modo Claro:** preto sólido `#0A0A0A` (100% de opacidade) — sombra brutalista clássica, visível sobre o fundo osso.
>
> Deslocamentos: `4px 4px 0px 0px` (normal) e `6px 6px 0px 0px` (hover), nos dois temas.

📊 Contraste (WCAG)

Combinação	Contraste	Resultado
`#CCFF00` sobre `#0A0A0A` (escuro)	16,9 : 1	✅ AAA

Combinação	Contraste	Resultado
`#CCFF00` sobre `#FFFFFF` (claro — **antigo**)	1,17 : 1	❌ Invisível
`#4D6600` sobre `#FFFFFF` (card claro)	6,5 : 1	✅ AA
`#4D6600` sobre `#F5F5F0` (fundo claro)	6,0 : 1	✅ AA
`#0A0A0A` sobre `#CCFF00` (texto em fill)	16,9 : 1	✅ AAA
`#525252` sobre `#F5F5F0` (muted claro)	≈ 7 : 1	✅ AAA
`#B91C1C` sobre `#FFFFFF` (destrutivo claro)	6,2 : 1	✅ AA
`#EF4343` sobre `#1A1A1A` (destrutivo escuro)	5,2 : 1	✅ AA

🎨 Tabela Geral de Códigos Hexadecimais (HEX) e Valores

Token / Papel Semântico	Variável CSS	HSL	HEX	RGB / RGBA
Verde Neon Principal (Fill)	`--primary`, `--accent`	`72 100% 50%`	**`#CCFF00`**	`rgb(204, 255, 0)`
Verde Tinta (Modo Claro)	`--primary-ink`	`75 100% 20%`	**`#4D6600`**	`rgb(77, 102, 0)`
Verde Tinta (Modo Escuro)	`--primary-ink`	`72 100% 50%`	**`#CCFF00`**	`rgb(204, 255, 0)`
Sombra dos Cards (Claro)	`--shadow` + `--shadow-alpha`	`75 100% 20%` / `0.35`	**`#4D6600`**	`rgba(77, 102, 0, 0.35)`
Sombra dos Cards (Escuro, Normal)	`--shadow` + `--shadow-alpha`	`72 100% 50%` / `0.2`	**`#CCFF00` (20% alfa)**	`rgba(204, 255, 0, 0.2)`
Sombra dos Cards (Escuro, Hover)	`--shadow` + `--shadow-alpha-hover`	`72 100% 50%` / `0.4`	**`#CCFF00` (40% alfa)**	`rgba(204, 255, 0, 0.4)`
Fundo da Página (Modo Claro)	`--background`	`60 20% 95%`	**`#F5F5F0`**	`rgb(245, 245, 240)`
Fundo da Página (Modo Escuro)	`--background`	`0 0% 4%`	**`#0A0A0A`**	`rgb(10, 10, 10)`
Fundo dos Cards (Modo Claro)	`--card`	`0 0% 100%`	**`#FFFFFF`**	`rgb(255, 255, 255)`
Fundo dos Cards (Modo Escuro)	`--card`	`0 0% 10%`	**`#1A1A1A`**	`rgb(26, 26, 26)`
Texto Principal (Modo Claro)	`--foreground`	`0 0% 4%`	**`#0A0A0A`**	`rgb(10, 10, 10)`
Texto Principal (Modo Escuro)	`--foreground`	`0 0% 100%`	**`#FFFFFF`**	`rgb(255, 255, 255)`
Texto Secundário / Muted (Claro)	`--muted-foreground`	`0 0% 32%`	**`#525252`**	`rgb(82, 82, 82)`
Texto Secundário / Muted (Escuro)	`--muted-foreground`	`0 0% 80%`	**`#CCCCCC`**	`rgb(204, 204, 204)`

Token / Papel Semântico	Variável CSS	HSL	HEX	RGB / RGBA
Bordas e Inputs (Modo Claro)	`--border`, `--input`	`75 55% 14%`	**`#2A3810`**	`rgb(42, 56, 16)`
Bordas e Inputs (Modo Escuro)	`--border`, `--input`	`0 0% 15%`	**`#262626`**	`rgb(38, 38, 38)`
Anel de Foco (Modo Claro)	`--ring`	`75 100% 20%`	**`#4D6600`**	`rgb(77, 102, 0)`
Anel de Foco (Modo Escuro)	`--ring`	`72 100% 50%`	**`#CCFF00`**	`rgb(204, 255, 0)`
Superfície de Apoio / Tag (Claro)	`--secondary`	`60 10% 88%`	**`#E3E3DC`**	`rgb(227, 227, 220)`
Superfície de Apoio / Tag (Escuro)	`--secondary`	`0 0% 15%`	**`#262626`**	`rgb(38, 38, 38)`
Vermelho Destrutivo (Modo Claro)	`--destructive`	`0 74% 42%`	**`#B91C1C`**	`rgb(185, 28, 28)`
Vermelho Destrutivo (Modo Escuro)	`--destructive`	`0 84% 60%`	**`#EF4343`**	`rgb(239, 67, 67)`
Texto sobre Destrutivo	`--destructive-foreground`	`0 0% 100%`	**`#FFFFFF`**	`rgb(255, 255, 255)`
Texto sobre Primário	`--primary-foreground`	`0 0% 4%`	**`#0A0A0A`**	`rgb(10, 10, 10)`

🔎 Detalhamento por Elemento do Diário

1. Cor Verde Principal (Primary / Accent / Neon)

- **Código HEX:** `#CCFF00`
- **RGB:** `rgb(204, 255, 0)`
- **HSL:** `hsl(72, 100%, 50%)`
- **Variante de tinta (modo claro):** `#4D6600` — `hsl(75, 100%, 20%)`, via `--primary-ink`.
- **Onde aparece no Diário e como aplicar:**

Elemento	Classe / Tratamento	Modo Claro	Modo Escuro
Logo `CALORIE SYNC`	"SYNC" como **marca-texto**: `bg-primary text-primary-foreground px-1`	Bloco neon, letra preta	Bloco neon, letra preta
Ícone de chama do cabeçalho	`text-primary-ink`	Oliva `#4D6600`	Neon `#CCFF00`
Subtítulo "PAINEL DE PERFORMANCE"	`text-primary-ink`	Oliva	Neon
Barras de progresso (Calorias, Proteínas, Carboidratos, Gorduras)	Trilho `bg-secondary`, fill `bg-primary`, `border-2 border-foreground` no claro	Neon com contorno preto	Neon
Ícones dos cards de métricas	`text-primary-ink`	Oliva	Neon

Elemento	Classe / Tratamento	Modo Claro	Modo Escuro
Badge de contagem de refeições	`bg-primary text-primary-foreground border-2 border-edge`	Neon + borda preta	Neon + borda preta
Botão **"NOVA REFEIÇÃO"**	`bg-primary text-primary-foreground border-2 border-edge shadow-hard`	Neon, borda e sombra pretas	Neon, sombra neon
Linha de realce do cabeçalho	`bg-primary` + `border-b-2 border-foreground`	Faixa neon com base preta	Faixa neon
Foco de teclado	`ring-ring` / `focus:border-ring`	Preto	Neon
Seleção de texto (`::selection`)	Fundo `--primary`, texto `--primary-foreground`	Neon + preto	Neon + preto

---

2. Sombra dos Cards e Botões (`box-shadow`)

O CalorieSync segue a estética **Brutalista** (bordas rígidas retas sem raio de curvatura `rounded-none`, bordas firmes de 2px e sombras duras sem desfoque/blur).

A. Sombra Padrão dos Cards (`shadow-hard`) — agora adaptativa ao tema

Definida em `frontend/tailwind.config.js`, consumindo variáveis CSS:

```
box-shadow: 4px 4px 0px 0px hsl(var(--shadow) / var(--shadow-alpha));
```

- **Modo Claro:** preto `#0A0A0A` a 100% → `4px 4px 0px 0px rgba(10, 10, 10, 1)`.
- **Modo Escuro:** verde neon `#CCFF00` a 20% → `4px 4px 0px 0px rgba(204, 255, 0, 0.2)`.
- **Hover (`shadow-hard-hover`):**

  ```
  box-shadow: 6px 6px 0px 0px hsl(var(--shadow) / var(--shadow-alpha-hover));
  ```

  - **Modo Claro:** permanece preto sólido, expandindo 2px.
  - **Modo Escuro:** neon sobe para 40% de opacidade, expandindo 2px.
- **Utilizada em:**
  - Cards de métricas diárias (Calorias, Proteínas, Carboidratos, Gorduras).
  - Navegador diário de datas ("Dia anterior / Próximo dia").
  - Card de chamada para ação rápida ("Registrar Refeição").
  - Cards individuais de cada refeição registrada (`hover:shadow-hard`).
  - Modais (modal de nova refeição, confirmação de exclusão e cópia).

B. Sombra Brutalista de Alto Contraste (Elementos com fill neon)

Para elementos que já têm o fundo totalmente verde neon (como abas selecionadas ou avatares), uma sombra neon transparente perderia o contraste. Por isso, utiliza-se a sombra brutalista neutra:

- **Modo Claro:** `4px 4px 0px 0px rgba(0, 0, 0, 1)` (Preto puro `#000000`) — agora coincide com o `shadow-hard` padrão.
- **Modo Escuro:** `4px 4px 0px 0px rgba(255, 255, 255, 0.2)` (Branco `#FFFFFF` a 20%).

---

3. Fundo da Página (`--background`)

- **Modo Claro:**
  - **HEX:** `#F5F5F0`
  - **RGB:** `rgb(245, 245, 240)`
  - **Sensação:** Off-white "osso"/papel. Tira o branco hospitalar e cria separação clara para os cards brancos com contorno preto.
- **Modo Escuro:**
  - **HEX:** `#0A0A0A`
  - **RGB:** `rgb(10, 10, 10)`
  - **Sensação:** Preto profundo/chumbo ("Deep Black"), ideal para destacar o verde neon `#CCFF00`.

---

4. Fundo dos Cards (`--card`)

- **Modo Claro:**
  - **HEX:** `#FFFFFF`
  - **RGB:** `rgb(255, 255, 255)`
  - **Diferença para o fundo:** Branco puro que "salta" do fundo osso `#F5F5F0`, reforçado pela borda preta de 2px e pela sombra preta.
- **Modo Escuro:**
  - **HEX:** `#1A1A1A`
  - **RGB:** `rgb(26, 26, 26)`
  - **Diferença para o fundo:** Um tom de cinza escuro que se eleva visualmente em relação ao fundo preto `#0A0A0A`.

---

5. Bordas dos Cards e Divisórias (`--border`)

- **Espessura Obrigatória:** `border-2` (2 pixels de contorno sólido).
- **Modo Claro:** `#0A0A0A` (`rgb(10, 10, 10)`) — contorno preto, brutalismo "de tinta".
- **Modo Escuro:** `#262626` (`rgb(38, 38, 38)`).
- **Fills neon:** use `border-edge` (verde-oliva escuro no claro, preto no escuro).

---

6. Tipografia e Textos

- **Texto Principal (`--foreground`):**
  - **Modo Claro:** `#0A0A0A` (Preto quase absoluto).
  - **Modo Escuro:** `#FFFFFF` (Branco puro).
- **Texto Secundário / Legendas / Labels (`--muted-foreground`):**
  - **Modo Claro:** `#525252` (Cinza escuro, ≈ 7 : 1 sobre o fundo osso).
  - **Modo Escuro:** `#CCCCCC` (Cinza claro).
- **Texto/Ícone em verde (`--primary-ink`):**
  - **Modo Claro:** `#4D6600` (Verde-oliva).
  - **Modo Escuro:** `#CCFF00` (Verde neon).

- **Texto sobre o Botão Verde Primário (`--primary-foreground`):**
  - Sempre `#0A0A0A` (Preto absoluto) para leitura nítida e contraste acessível (WCAG AAA) sobre o verde neon.

---

7. Ações Destrutivas e Erros (`--destructive`)

- **Código HEX:** `#EF4343`
- **RGB:** `rgb(239, 67, 67)`
- **Fundo com transparência (Alertas e banners):** `bg-destructive/10` (`rgba(239, 67, 67, 0.1)`).
- **Onde aparece no Diário:**
  - Botão e ícone de lixeira para exclusão de refeição.
  - Modal de confirmação "EXCLUIR REFEIÇÃO".
  - Banner de erro quando as refeições falham em carregar.

💻 Como as Cores São Configuradas no Código

1. **Tokens Semânticos (Variáveis CSS):**
   - Arquivo: `frontend/src/styles/index.css`
   - O Tailwind consome os valores HSL através da função `hsl(var(--token) / <alpha-value>)`.

```
:root {
  --background: 60 20% 95%;        /* #F5F5F0 */
  --foreground: 0 0% 4%;           /* #0A0A0A */
  --card: 0 0% 100%;               /* #FFFFFF */
  --card-foreground: 0 0% 4%;
  --primary: 72 100% 50%;          /* #CCFF00 — fill */
  --primary-foreground: 0 0% 4%;
  --primary-ink: 75 100% 20%;      /* #4D6600 — texto/ícones */
  --secondary: 60 10% 88%;         /* #E3E3DC */
  --muted-foreground: 0 0% 32%;    /* #525252 */
  --border: 0 0% 4%;               /* contorno preto */
  --input: 0 0% 4%;
  --ring: 0 0% 4%;

  --destructive: 0 74% 42%;        /* #B91C1C — claro */
  --destructive-foreground: 0 0% 100%;

  --warning: 38 92% 50%;           /* #F59E0B — avisos e favoritos */
  --warning-foreground: 0 0% 4%;

  /* Sombra brutalista */
  --shadow: 0 0% 4%;
  --shadow-alpha: 1;
  --shadow-alpha-hover: 1;

  /* Sombra neutra para fills neon */
  --shadow-fill: 75 55% 14%;
  --shadow-fill-alpha: 0.85;
  --shadow-fill-alpha-hover: 1;
  --edge: 75 55% 14%;      /* border-edge: contorno de fills neon (preto no escuro) */
}

.dark {
  --background: 0 0% 4%;           /* #0A0A0A */
  --foreground: 0 0% 100%;         /* #FFFFFF */
  --card: 0 0% 10%;                /* #1A1A1A */
  --card-foreground: 0 0% 100%;
  --primary: 72 100% 50%;          /* #CCFF00 */
  --primary-foreground: 0 0% 4%;
  --primary-ink: 72 100% 50%;      /* #CCFF00 */
  --secondary: 0 0% 15%;           /* #262626 */
  --muted-foreground: 0 0% 80%;    /* #CCCCCC */
  --destructive: 0 84% 60%;        /* #EF4343 — escuro */
  --destructive-foreground: 0 0% 100%;
  --warning: 38 92% 50%;           /* #F59E0B */
  --warning-foreground: 0 0% 4%;
  --border: 0 0% 15%;              /* #262626 */
  --input: 0 0% 15%;
  --ring: 72 100% 50%;             /* #CCFF00 */

  --shadow: 72 100% 50%;
  --shadow-alpha: 0.2;
  --shadow-alpha-hover: 0.4;

  --shadow-fill: 0 0% 100%;
  --shadow-fill-alpha: 0.2;
  --shadow-fill-alpha-hover: 0.4;
}

::selection {
  background: hsl(var(--primary));
  color: hsl(var(--primary-foreground));
}
```

2. **Cores e Sombras Brutalistas no Tailwind:**
    - Arquivo: `frontend/tailwind.config.js` sob `theme.extend`:

```
colors: {
  primary: {
    DEFAULT: 'hsl(var(--primary) / <alpha-value>)',
    foreground: 'hsl(var(--primary-foreground) / <alpha-value>)',
    ink: 'hsl(var(--primary-ink) / <alpha-value>)', // text-primary-ink
  },
  warning: {
    DEFAULT: 'hsl(var(--warning) / <alpha-value>)',
    foreground: 'hsl(var(--warning-foreground) / <alpha-value>)',
  },
},
boxShadow: {
  hard: '4px 4px 0px 0px hsl(var(--shadow) / var(--shadow-alpha))',
  'hard-hover': '6px 6px 0px 0px hsl(var(--shadow) / var(--shadow-alpha-hover))',
  'hard-fill': '4px 4px 0px 0px hsl(var(--shadow-fill) / var(--shadow-fill-alpha))',
},
```

3. **Alternância de Tema:**
    - A classe `.dark` é injetada na tag `<html>`, trocando automaticamente as variáveis CSS de `:root` para `.dark` sem necessidade de classes redundantes no HTML.
4. **Checklist de Migração no Código:**
    - ☑ Substituir `text-primary` usado em **textos e ícones sobre fundo** por `text-primary-ink`.
    - ☑ Manter `bg-primary` nos fills (botões, barras, badges) e adicionar `border-2 border-edge` (ou `border-foreground`).
    - ☑ Trocar `focus:border-primary` / `ring-primary` por `focus:border-ring` / `ring-ring`.
    - ☑ Transformar o "SYNC" do logo em marca-texto (`bg-primary text-primary-foreground px-1`).
    - ☑ Verificar se nenhuma sombra está com `rgba(204, 255, 0, …)` fixo no código (usar `shadow-hard` / `shadow-hard-hover`).

💠 Exceção Documentada: Vidro Translúcido da Barra Inferior

O "sem blur" deste guia vale para **sombras**. A barra inferior mobile (`BottomNav`) é a única superfície com translucidez: `bg-background supports-[backdrop-filter]:bg-background/85 backdrop-blur-2xl backdrop-saturate-150`, com `border-t border-border/40` (borda superior fina, exceção consciente ao `border-2`). Sem `backdrop-filter` o fundo continua sólido (`bg-background`). Só tokens, com opacidade em token; sem hex/rgba, sem sombra e sem `rounded`.

🛡️ Como Evitar Regressão

Para garantir que novos componentes e refatorações preservem a integridade da paleta semântica e do contraste WCAG:

1. **Guarda Automática de Cores (`check:colors`):**
    - Integrado ao pipeline de testes e ao lint raiz (`npm run lint`), o script `frontend/scripts/check-colors.mjs` verifica todos os arquivos do frontend contra

padrões proibidos:

- Literais hexadecimais (`#hex`) ou funções numéricas `rgb()` / `rgba()` / `hsl()` (exceto `hsl(var(--token))`).
- Classes `text-primary` ou `text-accent` soltas (sem sufixo `-ink` ou `-foreground`).
- Utilitários da paleta padrão do Tailwind com números (ex.: `gray-500`, `blue-600`).
- Variantes `dark:` aplicadas diretamente a classes utilitárias de cor (as cores se adaptam automaticamente via tokens CSS).
- Valores arbitrários como `bg-[#...]` ou `shadow-[...]`.

2. **Exceção Pontual Controlada:**
   - Se um elemento estritamente necessitar de uma exceção, use o comentário `/* color-guard-allow: <motivo> */` ou `// color-guard-allow: <motivo>` na mesma linha ou na anterior.
3. **Preenchimento vs. Tinta:**
   - **`bg-primary`** é apenas para **superfície** (botão, badge, progresso), acompanhado de `text-primary-foreground` e `border-2 border-edge`.
   - **`text-primary-ink`** é para **tinta** (textos, ícones, traços).
4. **Foco Visível:**
   - Use `focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 focus-visible:ring-offset-background`.

Configuração de e-mail transacional (BL-12 / BL-05)

Guia operacional para confirmação de e-mail e recuperação de senha. O código usa apenas SMTP genérico (`nodemailer`), sem SDK de provedor. Em desenvolvimento, o **Mailpit** captura todas as mensagens; em produção, aponte as variáveis para o SMTP do provedor escolhido (Resend, SendGrid, Amazon SES, Postmark etc.).

---

1. Como o envio funciona

1. `backend/src/mail/mailer.ts` escolhe a implementação por `MAIL_TRANSPORT`:
   - `smtp`: `SmtpMailer` via `nodemailer`.
   - `memory`: `MemoryMailer`, caixa em memória usada pelos testes.
   - `http`: `HttpMailer`, `POST` JSON `{ secret, to, subject, text, html }` para `MAIL_HTTP_URL` (Web App do Google Apps Script; veja a seção 3.4).
   - `NODE_ENV=test` força `MemoryMailer`, independentemente de `MAIL_TRANSPORT`.
2. Os fluxos de cadastro, reenvio, login não verificado e recuperação **enviam sem `await`** depois do commit. A resposta HTTP nunca depende do provedor de e-mail.
3. Falhas de entrega são registradas como `auth_email_delivery_failed`, sem código, token, URL de reset ou senha.
4. Nenhum código ou token é gravado em texto puro:
   - Código de verificação: HMAC-SHA256 com `AUTH_TOKEN_PEPPER`.
   - Token de reset: SHA-256.

---

2. Desenvolvimento local com Mailpit

O `docker-compose.yml` sobe o PostgreSQL e o Mailpit:

```
npm run docker:up
```

Serviço	Endereço
SMTP	`localhost:1025` (sem autenticação)
Interface web	http://localhost:8025

Configuração correspondente em `backend/.env`:

```
APP_BASE_URL=http://localhost:5173
MAIL_FROM="CalorieSync <no-reply@caloriesync.local>"
MAIL_TRANSPORT=smtp
MAIL_SMTP_HOST=localhost
MAIL_SMTP_PORT=1025
MAIL_SMTP_USER=
MAIL_SMTP_PASS=
MAIL_SMTP_SECURE=false
```

Depois:

```
npm run dev:all
```

Fluxo de teste manual:

1. Abra http://localhost:5173/cadastro e crie uma conta. A resposta é `202` genérica e **não** abre sessão.
2. Abra http://localhost:8025 e localize a mensagem com o código de 6 dígitos.
3. Informe o código em `/verificar-email`. O login é liberado e a sessão é criada.
4. Saia e use "Esqueci minha senha". O Mailpit recebe o link `http://localhost:5173/redefinir-senha#token=...`.
5. Abra o link, defina a nova senha e entre novamente. O token é de uso único e expira em 30 min.

Para trabalhar sem SMTP, use `MAIL_TRANSPORT=memory`. A caixa é interna ao processo e não tem interface visual; os testes a consultam pelo helper `lastEmailTo`.

---

3. Produção com provedor SMTP (exemplo: Resend)

3.1 Preparar o domínio

1. Crie a conta no provedor e adicione o domínio remetente (ex.: `caloriesync.app`).
2. No painel do provedor, copie os registros DNS exibidos:
   - **SPF** (`TXT` no domínio ou subdomínio de envio).
   - **DKIM** (um ou mais `TXT`/`CNAME`, conforme o provedor).
   - **DMARC** (`TXT` em `_dmarc`, começando com `v=DMARC1; p=none;` e um endereço de relatórios).
3. Aguarde a propagação e confirme a verificação no painel. Sem DKIM/SPF alinhados, mensagens transacionais tendem a cair em spam ou serem rejeitadas.
4. Use um remetente do domínio verificado, por exemplo `MAIL_FROM="CalorieSync <no-reply@caloriesync.app>"`. Nunca use um endereço pessoal.

> Os valores exatos de SPF/DKIM variam por provedor e por região. Use sempre os registros exibidos no painel do seu provedor; os exemplos acima não substituem essa verificação.

3.2 Credenciais SMTP

No provedor, gere uma credencial SMTP e configure o backend por variáveis de ambiente (nunca no Git):

```
NODE_ENV=production
APP_BASE_URL=https://app.caloriesync.app
MAIL_FROM="CalorieSync <no-reply@caloriesync.app>"
MAIL_TRANSPORT=smtp
MAIL_SMTP_HOST=smtp.resend.com
MAIL_SMTP_PORT=587
MAIL_SMTP_USER=resend
MAIL_SMTP_PASS=<credencial-smtp-do-provedor>
MAIL_SMTP_SECURE=false
AUTH_TOKEN_PEPPER=<32+ bytes aleatórios, diferente de JWT_SECRET>
```

Observações:

- Porta `587` normalmente usa STARTTLS (`MAIL_SMTP_SECURE=false`). Porta `465` usa TLS direto (`MAIL_SMTP_SECURE=true`). Confirme no provedor.
- `APP_BASE_URL` deve ser a URL pública **HTTPS** do frontend. Ela compõe o link de reset.
- Gere o pepper com:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

- `AUTH_TOKEN_PEPPER` deve ter no mínimo 32 caracteres e ser **diferente** de `JWT_SECRET`. A validação de ambiente falha na inicialização se essa regra for violada.
- Guarde as credenciais no gerenciador de segredos do ambiente. Não commit `backend/.env`.

3.3 Timeouts e Segurança TLS em Produção (#14)

Para prevenir vazamento de conexões ou travamento de sockets ociosos no servidor, o transporte SMTP aplica os seguintes limites no `nodemailer`:

- **`connectionTimeout`:** `10000` (10 segundos para estabelecer handshake de rede).
- **`greetingTimeout`:** `10000` (10 segundos para receber a saudação SMTP do servidor).
- **`socketTimeout`:** `15000` (15 segundos para transmissão de dados antes de encerrar o socket ocioso).
- **`requireTLS`:** Forçado em `true` em ambiente de produção (`NODE_ENV=production`) quando `MAIL_SMTP_SECURE=false` (portas 587/25 com STARTTLS), garantindo canal seguro obrigatório.
- **Proibição de `memory`:** A inicialização do backend encerra imediatamente (`fail-fast`) se `MAIL_TRANSPORT=memory` for configurado com `NODE_ENV=production`.

3.3 Outros provedores

Provedor	Host SMTP usual	Usuário
Resend	`smtp.resend.com`	`resend`
SendGrid	`smtp.sendgrid.net`	`apikey`
Amazon SES	endpoint da região (ex.: `email-smtp.us-east-1.amazonaws.com`)	credencial SMTP IAM
Postmark	`smtp.postmarkapp.com`	token do servidor

Confirme portas, usuário e política de TLS no painel do provedor.

3.4 Envio por HTTPS (Google Apps Script, `MAIL_TRANSPORT=http`)

Usado em hospedagem que bloqueia SMTP (Render grátis). O passo a passo do script está em HOSPEDAGEM.md, passo 5.

```
MAIL_TRANSPORT=http
MAIL_HTTP_URL=https://script.google.com/macros/s/<ID>/exec
MAIL_HTTP_SECRET=<mesmo valor da propriedade MAIL_SECRET do script>
```

- **Validação no boot:** `MAIL_HTTP_URL` e `MAIL_HTTP_SECRET` são obrigatórias com `http`. Em produção a URL precisa ser `https` e não pode ser `localhost`/`127.0.0.1`. O segredo exige no mínimo 32 caracteres e é recusado se parecer placeholder.

- **Segredo no corpo:** o Apps Script não lê headers, então o segredo vai no JSON.
- **Redirect:** o `/exec` responde `302` para `script.googleusercontent.com`; o `fetch` segue o redirect (o `POST` vira `GET`, que é o esperado).
- **Sucesso:** só quando a resposta final é `2xx` com JSON `{ "ok": true }`. HTML, `ok:false`, status diferente de `2xx` e JSON inválido viram erro; o timeout é de 10 s.
- **Logs:** as mensagens de erro são fixas (`mail_http_timeout`, `mail_http_request_failed`, `mail_http_status_<n>`, `mail_http_invalid_response`, `mail_http_rejected`) e nunca trazem URL, segredo, destinatário, código ou link.

---

4. Segurança e operação

- **Nunca logue** código de verificação, token de reset, URL de reset ou senha. Os logs de falha têm apenas o nome do evento e a mensagem do erro.
- **Não armazene** código/token no `localStorage`. O e-mail de verificação fica no state do router e o token de reset no fragmento da URL, removido com `history.replaceState`.
- **Rate limits:** 5 envios de código por hora por e-mail, cooldown de 60 s, 5 resets por hora por e-mail, 10 tentativas de verificação por 15 min por IP e 10 resets por hora por IP.
- **Rotação do pepper:** invalida códigos de verificação pendentes. Tokens de reset usam SHA-256 e não são afetados, mas podem ser invalidados solicitando um novo link.
- **Bounces e reclamações:** acompanhe no painel do provedor. O produto não faz retentativa automática; uma falha de entrega é apenas registrada.
- **Cache:** todas as respostas de `/api/auth/*` levam `Cache-Control: no-store`.
- **Enumeração:** cadastro, reenvio e recuperação respondem sempre a mesma mensagem genérica, exista ou não a conta. Não altere esse comportamento em telas ou mensagens de erro.

---

5. Diagnóstico rápido

Sintoma	Verifique
Nenhum e-mail no Mailpit	`MAIL_TRANSPORT=smtp`, `MAIL_SMTP_HOST=localhost`, `MAIL_SMTP_PORT=1025`, container `caloriesync_mailpit` ativo
E-mail em spam em produção	SPF, DKIM e DMARC verificados; `MAIL_FROM` usa domínio verificado
Login retorna 403 `EMAIL_NOT_VERIFIED`	Conta ainda não confirmou o e-mail; abra `/verificar-email` e use o código mais recente
Código recusado	Expirou (15 min), já foi usado, atingiu 5 tentativas ou um código mais novo foi emitido
Link de reset inválido	Expirou (30 min), já foi usado ou um novo pedido invalidou o anterior
`APP_BASE_URL` errada	O link do e-mail aponta para o host configurado; em produção use HTTPS
Backend não inicia	`AUTH_TOKEN_PEPPER` ausente/curto ou igual a `JWT_SECRET`; `MAIL_FROM` inválido; `MAIL_HTTP_URL`/`MAIL_HTTP_SECRET` ausentes ou inválidos com `MAIL_TRANSPORT=http`

Sintoma	Verifique
`auth_email_delivery_failed` com `mail_http_rejected`	`MAIL_HTTP_SECRET` diferente do `MAIL_SECRET` do script
`auth_email_delivery_failed` com `mail_http_invalid_response`	Script implantado sem acesso "Qualquer pessoa" (o Google devolve página HTML de login)

---

6. Validação automatizada

```
npm run test:strict
npm --prefix frontend test -- --run
npm run lint
npm run build
npm --prefix frontend run check:colors
```

As suítes de BL-12/BL-05 usam `MemoryMailer` e o banco `contador_calorias_test`. Elas cobrem código expirado, tentativas, reenvio, cooldown, reset de uso único, revogação por `token_version` e respostas sem enumeração.

Plano de hospedagem gratuita do CalorieSync

Guia para quem nunca fez um deploy. Objetivo: colocar o site no ar **de graça**, de forma **confiável** e **simples**, para usar no portfólio, e saber como atualizá-lo depois.

> Os planos gratuitos mudam. Os números abaixo foram conferidos em 02/11/2026; antes de cada cadastro, abra o link da seção "Fontes" e confirme os limites.

---

1. O que é "hospedar" (em 2 minutos)

Hoje o site só funciona no seu computador (`localhost`). Hospedar é colocar as mesmas peças em computadores de outras empresas, ligados 24h e acessíveis por um endereço público (`https://...`).

O CalorieSync tem **3 peças**, e cada uma vai para um lugar:

Peça	O que é	Onde fica no projeto	Onde hospedar (grátis)
Site (front)	Arquivos HTML/JS/CSS que rodam no navegador (React + Vite)	`frontend/`	**Netlify**
API (back)	Programa Node.js/Express que aplica as regras e fala com o banco	`backend/` (+ `shared/`)	**Render**
Banco	PostgreSQL com usuários, refeições, catálogo de alimentos	migrations em `backend/migrations/`	**Neon**

Além delas, o cadastro envia e-mails (código de verificação, redefinir senha), então precisamos de um **serviço de e-mail**: o seu próprio **Gmail**, acionado por um **Google Apps Script** (grátis, sem domínio). Também obrigatório e grátis: **Cloudflare Turnstile** (captcha; a API não sobe em produção sem ele). Opcional: **UptimeRobot** (mantém a API acordada).

```
Navegador ⟶ Netlify (site + proxy /api) ⟶ Render (API) ⟶ Neon (Postgres)
                                                  └⟶ Google Apps Script ⟶ Gmail (e-mails, via
HTTPS)
```

Por que essa combinação

- **Netlify**: o front já tem `frontend/public/_headers` (CSP, HSTS), formato que o Netlify lê. Plano grátis sem prazo de validade.
- **Render**: roda o `Dockerfile` da API que já existe na raiz. Plano grátis sem cartão.
- **Neon**: Postgres grátis **sem prazo de validade** (o Postgres grátis do Render **expira em 30 dias** e é apagado, por isso não usamos). Suporta as extensões `pg_trgm`, `unaccent` e `uuid-ossp` que o projeto usa.
- **Gmail + Apps Script**: grátis e sem domínio. O e-mail sai dos servidores do próprio Google, então chega na caixa de entrada, e a API fala com ele por HTTPS, que o Render grátis permite (o Render grátis **bloqueia SMTP** desde set/2025, por isso Brevo/SMTP não funcionaria lá).

A regra que obriga a usar o Netlify como "proxy"

A sessão do usuário fica num cookie `cs_session` com `SameSite=Strict` (issue #1 em `docs/ISSUES.md`). Esse cookie **só é enviado se o site e a API parecerem o mesmo endereço**. Se o site estiver em `x.netlify.app` e a API em `y.onrender.com`, o login não funciona.

Solução sem mexer na lógica: o Netlify repassa tudo que começa com `/api/` para a API no Render. Para o navegador, tudo é `x.netlify.app`. Isso exige dois pequenos arquivos de configuração (passo 6).

---

2. Limites reais do plano grátis (leia antes)

Serviço	Limite	Efeito para você
Render (API)	Dorme após 15 min sem acesso; acordar leva até ~1 min. 750 horas/mês grátis. Sem disco persistente.	A **primeira visita** depois de um tempo parado fica lenta. Veja a mitigação no passo 9.
Neon (banco)	Sem prazo; ~0,5 a 1 GB por projeto (confira); o computador do banco "hiberna" e acorda sozinho.	Basta para o projeto; o primeiro acesso após hibernar demora um pouco.
Netlify (site)	Plano grátis com limite mensal de banda e builds.	Sobra para portfólio.
Render (API), e-mail	Bloqueia saída SMTP (portas 25, 465, 587) no plano grátis.	Por isso o e-mail sai por HTTPS (Apps Script), não por SMTP.
Gmail via Apps Script	~100 destinatários/dia (conta Gmail comum).	Sobra para portfólio: cada cadastro gasta 1 e-mail (2 se pedir reenvio).

Quem for avaliar seu portfólio pode pegar a API "dormindo". Coloque no README do projeto: *"A API gratuita pode levar ~1 min para acordar na primeira visita."*

Domínio próprio é opcional e pago (~R$ 40/ano). Neste plano usamos os endereços grátis `*.netlify.app` e `*.onrender.com`, e o e-mail sai de um Gmail, sem domínio.

---

3. Antes de começar (uma vez só)

1. Conta no **GitHub** com o repositório do projeto (já existe: `luiscorreiaa4/contador-de-calorias`). Tudo é publicado a partir dele.
2. Crie contas gratuitas, entrando com o GitHub quando houver a opção: Netlify, Render, Neon, Cloudflare (Turnstile) e UptimeRobot.
3. Instale na sua máquina o **cliente `psql`** (vem com o instalador do PostgreSQL; marque só "Command Line Tools"). Ele serve para rodar scripts SQL no Neon. Alternativa: usar o editor SQL do painel do Neon.
4. Abra um **bloco de notas seguro** (ou gerenciador de senhas) para guardar senhas e chaves. **Nunca** coloque segredos em arquivos commitados nem em chat.

Segredos que você vai gerar

Gere cada valor abaixo no terminal (cada um diferente):

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

- `JWT_SECRET` (mínimo 32 caracteres)
- `AUTH_TOKEN_PEPPER` (mínimo 32 caracteres, **diferente** do `JWT_SECRET`; a API recusa iniciar se forem iguais)
- Chaves de notificação push (VAPID): `npx web-push generate-vapid-keys` → `VAPID_PUBLIC_KEY` e `VAPID_PRIVATE_KEY` (sem elas, `GET /api/push/public-key` responde 503 e o botão de notificações não funciona; não há chave padrão)
- Senhas dos dois papéis de banco (`app_user` e `migration_user`), geradas do mesmo jeito

> **Nunca copie os valores do `backend/.env.example`.** A API recusa iniciar se `JWT_SECRET` ou `AUTH_TOKEN_PEPPER` contiverem `preencha`, `sua_senha`, `changeme`, `placeholder`, `example` ou `exemplo` (esses textos estão no repositório público; quem os conhecesse poderia forjar sessões, inclusive de admin). Use sempre o comando acima.

---

4. Passo a passo

A ordem importa, porque cada serviço precisa do endereço do outro.

Passo 1 · Reservar o nome do site no Netlify

1. No Netlify: **Add new site** → **Import an existing project** → **GitHub** e escolha o repositório.
2. **Não** publique ainda (ou deixe falhar). Em **Site configuration** → **Change site name**, use `caloriesync-luiscorreia` (é o nome que o repositório já usa no SEO). O endereço será `https://caloriesync-luiscorreia.netlify.app`. Em **Build settings**, deixe **Base directory vazio** (o `netlify.toml` está na raiz).
3. Anote esse endereço. Vamos chamá-lo de **`URL_SITE`**.

Passo 2 · Criar o banco no Neon

1. No Neon: **Create project**, nome `caloriesync`, versão do Postgres 16, região mais próxima do Render (ex.: `us-east`).
2. Na tela do projeto, clique em **Connect** e copie a **connection string** (formato `postgresql://usuario:senha@host/neondb?sslmode=require`). Chame-a de **`URL_ADMIN`**; ela tem poder total, **não** vai para a API.
3. As extensões `uuid-ossp`, `unaccent` e `pg_trgm` são criadas pelas migrations; o Neon permite isso.

Passo 3 · Criar as tabelas e os dados (migrations)

As migrations criam tabelas e já carregam o catálogo de alimentos. Rode **do seu computador**, apontando para o Neon:

```
$env:DATABASE_URL="<URL_ADMIN>"
npm run setup
```

Resultado esperado: lista de migrations aplicadas sem erro. Confira no painel do Neon (aba **Tables**) se `users`, `foods`, `meals` etc. existem. Feche o terminal depois (ou rode `Remove-Item Env:DATABASE_URL`) para não deixar a URL na sessão.

Passo 4 · Separar os papéis de banco (conclui o item #9)

Em vez de a API usar o usuário todo-poderoso, ela usa `app_user`, que só lê e grava dados (sem criar/apagar tabelas, sem editar o log de auditoria). Quem altera o esquema é o `migration_user`.

1. Rode o script do repositório como administrador, **depois** das migrations (ele precisa que `admin_audit_log` já exista):

   ```
   psql "<URL_ADMIN>" -v ON_ERROR_STOP=1 `
     -v app_password="'<senha_app>'" -v migration_password="'<senha_migracao>'" `
     -v dbname=neondb -f backend/scripts/provision-db-roles.sql
   ```

   (`dbname` é o nome do banco que aparece na connection string, normalmente `neondb`.) No Neon o dono (`neondb_owner`) não é superusuário. O script já lida com isso: só mexe em `NOSUPERUSER/NOCREATEDB/NOCREATEROLE` quando preciso e faz `GRANT migration_user TO neondb_owner;` antes do `ALTER DEFAULT PRIVILEGES` (sem esse grant, o Neon responde "permission denied to change default privileges").
2. Passe a posse das tabelas ao `migration_user`. **Não use `REASSIGN OWNED`**: no Neon ele falha porque tenta transferir o próprio banco. Rode com a `URL_ADMIN` (`psql "<URL_ADMIN>" -v ON_ERROR_STOP=1` e cole):

   ```
   DO $$
   DECLARE r record;
   BEGIN
     FOR r IN SELECT tablename FROM pg_tables
              WHERE schemaname = 'public' AND tableowner <> 'migration_user' LOOP
       EXECUTE format('ALTER TABLE public.%I OWNER TO migration_user', r.tablename);
     END LOOP;
     -- Sequences de serial/identity mudam junto com a tabela; aqui ficam as avulsas.
     FOR r IN SELECT sequencename FROM pg_sequences
              WHERE schemaname = 'public' AND sequenceowner <> 'migration_user' LOOP
       EXECUTE format('ALTER SEQUENCE public.%I OWNER TO migration_user', r.sequencename);
     END LOOP;
   END $$;
   ALTER FUNCTION public.f_unaccent(text) OWNER TO migration_user;
   ```

3. Monte as duas URLs que serão usadas depois (troque o host pelo da sua string do Neon):
   - **`URL_MIGRACAO`** = `postgresql://migration_user:<senha_migracao>@<host>/neondb?sslmode=require`
   - API: `DB_HOST=<host>`, `DB_PORT=5432`, `DB_USER=app_user`, `DB_PASSWORD=<senha_app>`, `DB_NAME=neondb`
4. **Teste** (esperado: dois `permission denied` e uma contagem):

   ```
   psql "postgresql://app_user:<senha_app>@<host>/neondb?sslmode=require" -c "CREATE TABLE x(i
   int);"
   psql "postgresql://app_user:<senha_app>@<host>/neondb?sslmode=require" -c "UPDATE
   admin_audit_log SET actor_type='x';"
   psql "postgresql://app_user:<senha_app>@<host>/neondb?sslmode=require" -c "SELECT count(*)
   FROM users;"
   ```

5. Dali em diante, **todas** as migrations futuras usam `URL_MIGRACAO`, nunca a URL de admin.

> Dev e testes locais continuam com o usuário `postgres` do Docker. O script só é para o banco hospedado. Backup (opcional, pois o banco está vazio agora): `backend/scripts/backup-db.mjs`.

Passo 5 · Configurar o e-mail (Gmail + Google Apps Script, grátis)

O cadastro só termina com o código de 6 dígitos enviado por e-mail, e "Esqueci minha senha" depende do link enviado por e-mail. Se o e-mail não chegar, ninguém consegue usar o site.

Por que este caminho:

- **O Render grátis bloqueia SMTP** (portas 25, 465 e 587) desde setembro de 2025. Qualquer serviço via SMTP (Brevo, Gmail SMTP, SendGrid) fica sem resposta lá; só HTTPS passa.
- **Um Gmail como remetente só chega na caixa de entrada se o envio sair do próprio Google.** Enviado por um terceiro (ex.: Brevo com remetente `@gmail.com`), falha na checagem DMARC e vai para o spam.
- O **Google Apps Script** resolve as duas coisas de graça: é um pequeno programa na sua conta Google com um endereço HTTPS. A API manda o e-mail para esse endereço e o script envia pelo seu Gmail. Limite de ~100 destinatários/dia.

> A API já tem o adaptador `MAIL_TRANSPORT=http` (`HttpMailer` em `backend/src/mail/mailer.ts`): faz `POST` de `{ secret, to, subject, text, html }` para `MAIL_HTTP_URL` e só considera enviado se o script responder `{"ok":true}`. `MAIL_HTTP_URL`/`MAIL_HTTP_SECRET` são validadas ao iniciar (`backend/src/config/env.ts`). Em desenvolvimento nada muda: o Mailpit continua com `MAIL_TRANSPORT=smtp`. Detalhes em `docs/EMAIL_SETUP.md` (seção 3.4).

5.1 Conta Gmail do projeto

1. Crie um Gmail só para o projeto (ex.: `caloriesync.app@gmail.com`). É o remetente que os usuários verão, e assim o seu e-mail pessoal fica de fora.
2. Ative a **verificação em duas etapas** nessa conta, porque o script terá permissão para enviar e-mails por ela.

5.2 Criar o script

1. Logado nessa conta, abra script.google.com → **Novo projeto** e dê o nome `CalorieSync Mailer`.
2. Apague o conteúdo de `Código.gs` e cole:

```
function doPost(e) {
  const secret = PropertiesService.getScriptProperties().getProperty('MAIL_SECRET');
  let body;
  try { body = JSON.parse(e.postData.contents); } catch (err) { return reply(false); }
  if (!secret || body.secret !== secret) return reply(false);
  MailApp.sendEmail({ to: body.to, subject: body.subject, body: body.text, htmlBody:
body.html, name: 'CalorieSync' });
  return reply(true);
}
function reply(ok) {
  return ContentService.createTextOutput(JSON.stringify({ ok: ok
})).setMimeType(ContentService.MimeType.JSON);
}
```

3. Gere um segredo com o mesmo comando da seção 3 (`randomBytes(32)`). Em **Configurações do projeto (engrenagem) → Propriedades do script → Adicionar propriedade**, crie `MAIL_SECRET` com esse valor. O segredo fica fora do código.
4. **Implantar → Nova implantação** → tipo **App da Web**. "Executar como": **Eu**. "Quem pode acessar": **Qualquer pessoa** (quem não tem o segredo recebe `{"ok":false}` e nada é enviado). Clique em **Implantar**.
5. O Google pede autorização. Como o app é seu e não verificado, aparece "O Google não verificou este app": clique em **Avançado → Acessar CalorieSync Mailer (não seguro) → Permitir**. É esperado; a permissão é só para enviar e-mail como você.
6. Copie a **URL do app da Web** (termina em `/exec`). Ela e o segredo vão no Render no passo 7 (`MAIL_HTTP_URL` e `MAIL_HTTP_SECRET`). Guarde os dois no bloco de notas seguro.

> Mudou o script depois? **Implantar → Gerenciar implantações → editar (lápis) → Versão: Nova versão**, e a URL continua a mesma. Uma "Nova implantação" gera outra URL.

5.3 Testar depois do deploy (passo 11)

1. Cadastre uma conta com outro Gmail e outra com um Outlook/Hotmail. O código deve chegar em até 1 min, **na caixa de entrada**, vindo do Gmail do projeto.
2. Teste "Esqueci minha senha": o link precisa começar com o seu `URL_SITE` em `https://` (vem de `APP_BASE_URL`) e abrir a tela de nova senha.
3. Se não chegar:
   - **Logs do Render**: falhas aparecem como `auth_email_delivery_failed`, sem dados sensíveis.
   - **Execuções** do Apps Script (menu à esquerda): cada pedido aparece ali, com o erro se houver (ex.: cota diária estourada).
   - Pasta **Enviados** do Gmail do projeto: mostra o que de fato saiu.

> **Se um dia passar de ~100 e-mails/dia:** aí vale um domínio próprio com um provedor transacional por API HTTPS (ex.: Brevo, Resend). O adaptador `http` só muda de destino. Para portfólio, não precisa.

Passo 6 · Configuração do Netlify (já no repositório)

O arquivo `netlify.toml` fica **na raiz do repositório** (não em `frontend/`), porque o build do front precisa do pacote `shared/` (`@caloriesync/shared`). No painel do Netlify (**Site configuration → Build & deploy → Build settings**) deixe **Base directory vazio**; comando e pasta publicada vêm do arquivo:

```
[build]
  command = "npm --prefix shared ci && npm --prefix frontend ci && npm --prefix frontend run build"
  publish = "frontend/dist"

[build.environment]
  NODE_VERSION = "24"
  VITE_API_URL = "/api"

# O front fala com "/api"; o Netlify repassa para a API no Render.
[[redirects]]
  from = "/api/*"
  to = "https://caloriesync-api.onrender.com/api/:splat"
  status = 200
  force = true

# SPA: qualquer outra rota abre a casca vazia spa.html (React Router).
# Sem force: "/", "/termos", "/privacidade" e "/seguranca" já existem como HTML pré-renderizado e têm prioridade.
[[redirects]]
  from = "/*"
  to = "/spa.html"
  status = 200
```

O proxy aponta para `caloriesync-api.onrender.com`. Se o nome do serviço no Render for outro (passo 7), troque no `netlify.toml` e faça commit.

E, no painel do Netlify (**Site configuration** → **Environment variables**), defina:

- `VITE_CAPTCHA_SITE_KEY` = site key do Turnstile (passo 7a)

Endereço do site (SEO). As páginas públicas já usam `https://caloriesync-luiscorreia.netlify.app` (`SITE_URL` em `frontend/src/seo/pages.ts`, `robots.txt`, `sitemap.xml` e `llms.txt` em `frontend/public/`). Se o nome do site no Netlify (passo 1) for outro, troque nesses 4 arquivos (sem barra final) e confira com `git grep caloriesync-luiscorreia`.

Confirmar o texto "gratuito". A landing (FAQ) e o JSON-LD (`offers` com preço 0 em `frontend/src/seo/pages.ts`) dizem que o CalorieSync é gratuito. Se um dia houver plano pago, ajuste os dois.

Pontos de atenção que eu **não verifiquei** e você deve testar no final:

- **Páginas pré-renderizadas:** o build gera `dist/index.html` (landing), `dist/termos/index.html`, `dist/privacidade/index.html` e `dist/seguranca/index.html` com o conteúdo já no HTML, e `dist/spa.html` (casca vazia) para o resto. O Netlify deveria servir `/termos` a partir de `termos/index.html` antes de aplicar o redirect `/*`; isso só foi simulado localmente. Teste no passo 11.
- Netlify pode cortar respostas de proxy lentas (~26 s). Com a API dormindo, a primeira chamada pode dar erro 502/504; recarregar resolve (o passo 9 evita).

Passo 7a · Criar o captcha (Cloudflare Turnstile), antes da API

Em produção a API **exige** `CAPTCHA_SECRET_KEY` e não sobe sem ela (proteção de cadastro, login e "esqueci a senha" contra robôs).

1. Cloudflare → **Turnstile** → **Add widget**, domínio = o de `URL_SITE` (ex.: `caloriesync-seunome.netlify.app`), modo **Managed** (invisível na maioria das visitas).

2. Anote a **Secret key** (vai para `CAPTCHA_SECRET_KEY` no Render, passo 7) e a **Site key** (vai para `VITE_CAPTCHA_SITE_KEY` no Netlify, passo 6).
3. As duas precisam ser do **mesmo widget**. Se o domínio do site mudar, edite o widget.
4. **Não** use as chaves de teste da Cloudflare (`1x0000…AA`) em produção: elas sempre aprovam.

Passo 7 · Publicar a API no Render

1. No Render: **New → Web Service → conecte o GitHub → escolha o repositório**.
2. Configure:
   - **Name:** `caloriesync-api` (o `netlify.toml` repassa `/api` para `https://caloriesync-api.onrender.com`)
   - **Language/Runtime:** Docker (usa o `Dockerfile` da raiz)
   - **Branch:** `main`
   - **Instance Type:** Free
   - **Health Check Path:** `/health`
   - **Auto-Deploy:** Yes (publica sozinho a cada push em `main`)
3. Em **Environment**, adicione as variáveis (cole os valores; marque como segredo o que for senha):

Variável	Valor
`NODE_ENV`	`production` (**obrigatória**: sem ela a API não sobe; antes, a ausência ligava o modo desenvolvimento e desligava cookie `Secure`, TLS do banco e as travas de CORS/e-mail)
`PORT`	`3001`
`DB_HOST` / `DB_PORT` / `DB_NAME`	do Neon (passo 4)
`DB_USER` / `DB_PASSWORD`	`app_user` / `<senha_app>`
`DATABASE_URL`	pode usar a URL do `app_user`; a API não migra em runtime
`JWT_SECRET`	gerado no passo "Segredos" (nunca o valor do `.env.example`)
`JWT_EXPIRES_IN`	`1d`
`AUTH_TOKEN_PEPPER`	gerado (diferente do JWT)
`CORS_ORIGIN`	`URL_SITE` (ex.: `https://caloriesync-seunome.netlify.app`, **sem barra final, https obrigatório**)
`APP_BASE_URL`	o mesmo `URL_SITE`
`TRUST_PROXY`	`1` (a API está atrás do proxy do Render; ver nota abaixo)
`MAIL_TRANSPORT`	`http` (passo 5; `smtp` não funciona no Render grátis e `memory` é proibido em produção)
`MAIL_FROM`	`CalorieSync <caloriesync.app@gmail.com>` (o Gmail do projeto)
`MAIL_HTTP_URL`	URL `/exec` do Apps Script (passo 5.2)
`MAIL_HTTP_SECRET`	o `MAIL_SECRET` do script (passo 5.2)
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY`	geradas no passo "Segredos" (sem a pública, push responde 503)

Variável	Valor
`CAPTCHA_SECRET_KEY`	secret do Turnstile (passo 7a); **obrigatória** em produção

A API valida tudo ao iniciar e **se recusa a subir** com valor faltando ou inseguro; a mensagem aparece em **Logs**. Em produção ela recusa, entre outros: `NODE_ENV` ausente, segredo placeholder, `JWT_SECRET` igual ao pepper, `CAPTCHA_SECRET_KEY` vazia, `MAIL_TRANSPORT=memory`, SMTP sem usuário/senha, `CORS_ORIGIN`/`APP_BASE_URL` sem `https` ou com `localhost`. O `Dockerfile` já define `NODE_ENV=production`, mas declare a variável no Render mesmo assim: se um dia a API rodar fora dessa imagem, ela continua protegida. 4. Clique em **Create Web Service**. Quando ficar "Live", anote o endereço (`https://<NOME_DA_API>.onrender.com`) e abra `/health`: deve responder `healthy`. 5. Se o nome não for `caloriesync-api`, troque o destino do `/api/*` no `netlify.toml` (raiz) pelo nome real. Faça commit e push para `main`.

> **Nota `TRUST_PROXY`:** o limite de tentativas de login usa o IP do visitante. Com Netlify → Render há mais de um salto e o IP real pode não chegar à API; o efeito pior é que muitos visitantes pareçam o mesmo IP e dividam o limite. Para portfólio é aceitável. Teste errando a senha 5 vezes: a conta deve bloquear, e outro usuário não deve ser afetado.

Passo 8 · Conferir o captcha

O widget foi criado no passo 7a. Depois do deploy do site (passo 9), confira:

1. `VITE_CAPTCHA_SITE_KEY` está no Netlify e o deploy foi feito **depois** de defini-la (variáveis `VITE_*` entram no build; mudar exige novo deploy).
2. Cadastro e login funcionam. Se aparecer "Falha na verificação de segurança" (`CAPTCHA_FAILED`), a site key e a secret não são do mesmo widget, ou o domínio do widget não é o do site.

Passo 9 · Publicar o site e manter a API acordada

1. No Netlify, com o `netlify.toml` já no GitHub, rode **Deploys** → **Trigger deploy**. Ao terminar, abra `URL_SITE`.
2. **UptimeRobot** → New monitor → HTTP(s) → URL `https://<NOME_DA_API>.onrender.com/health` → intervalo 5 min. Isso impede a API de dormir. 750 h/mês cobrem um serviço rodando 24h (31 dias = 744 h), então é seguro com **uma** API.

Passo 10 · Criar sua conta de administrador

O projeto proíbe criar admin por seed ou variável de ambiente. O caminho é:

1. No site, **cadastre-se normalmente** e confirme o e-mail com o código recebido.
2. No seu computador, aponte para o banco hospedado com as credenciais do **`app_user`** e promova a conta:

```
$env:NODE_ENV="production"
$env:DB_HOST="<host>"; $env:DB_PORT="5432"; $env:DB_NAME="neondb"
$env:DB_USER="app_user"; $env:DB_PASSWORD="<senha_app>"
# Demais variáveis obrigatórias da API (JWT_SECRET, AUTH_TOKEN_PEPPER, CORS_ORIGIN,
APP_BASE_URL, MAIL_*, CAPTCHA_SECRET_KEY): mesmos valores do Render.
npm run admin:promote -- seu@email.com --yes
```

Se a validação de ambiente reclamar de alguma variável, copie-a do Render.

3. Admin exige **MFA**: entre em Configurações → Segurança e ative antes de usar `/admin`.
4. Limpe as variáveis da sessão ao terminar.

Passo 11 · Checklist de "está funcionando?"

- [ ] `https://<API>.onrender.com/health` responde `healthy`
- [ ] Abrir `URL_SITE` carrega a landing e a rota `/login` (atualize a página com F5 em `/login` para testar o fallback)
- [ ] Logado, F5 em `/app/historico` abre o app (fallback `spa.html`); `/historico` sem o prefixo redireciona para `/app/historico/...`
- [ ] **Exibir código-fonte** (Ctrl+U) de `/`, `/termos`, `/privacidade` e `/seguranca` mostra o texto da página dentro de `<div id="root">`, o `<title>` próprio e o `<link rel="canonical">` com `URL_SITE` (se vier `<div id="root"></div>` vazio, o Netlify serviu `spa.html` no lugar do HTML pré-renderizado)
- [ ] F12 → Console em `/` e `/termos` sem erro do React (`#418` = hidratação divergente)
- [ ] `URL_SITE/robots.txt`, `URL_SITE/sitemap.xml`, `URL_SITE/llms.txt` e `URL_SITE/og-image.png` abrem e citam `URL_SITE`
- [ ] Prévia do link em redes sociais mostra a imagem 1200×630: teste no Facebook Sharing Debugger e no Teste de pesquisa aprimorada do Google (deve achar `SoftwareApplication` e `FAQPage` em `/`)
- [ ] O PWA instalado abre em `/app`
- [ ] Cadastro envia e-mail **na caixa de entrada** (Gmail e Outlook) e o código verifica (passo 5.3)
- [ ] "Esqueci minha senha" envia o link com `https://` + `URL_SITE` e a nova senha funciona
- [ ] Cadastro/login passam pelo captcha sem erro `CAPTCHA_FAILED`
- [ ] Logs do Render, no início, sem "Configuração de Ambiente Inválida"
- [ ] Login **sem** "lembrar de mim": o cookie `cs_session` aparece como "Session" em F12 → Application → Cookies; **com** a opção marcada, tem data de expiração (1 dia)
- [ ] Ativar notificações em Configurações funciona (se der erro, faltam as chaves VAPID no Render)
- [ ] Login mantém a sessão ao recarregar (se não manter, o proxy `/api` ou `VITE_API_URL=/api` está errado)
- [ ] Registrar uma refeição e buscar um alimento funciona
- [ ] No navegador (F12 → Application → Cookies) existe `cs_session` com `HttpOnly` e `Secure`
- [ ] Logs do Render sem `permission denied`
- [ ] Conta admin com MFA acessa `/admin`

Se algo falhar

Sintoma	Causa provável
API não sobe, log com "Configuração de Ambiente Inválida"	Variável faltando/inválida (CORS com barra, `APP_BASE_URL` sem https, pepper igual ao JWT). O log diz qual: `NODE_ENV é obrigatório`, `... parece um placeholder` (gere um segredo novo), `CAPTCHA_SECRET_KEY é obrigatório em produção` (passo 7a)
`CAPTCHA_FAILED` no cadastro/login	Site key (Netlify) e secret (Render) de widgets diferentes, domínio do widget errado, ou `VITE_CAPTCHA_SITE_KEY` definida depois do último deploy
Notificações: "Chave pública VAPID não configurada" (503)	Faltam `VAPID_PUBLIC_KEY`/`VAPID_PRIVATE_KEY` no Render
Login com MFA sempre falha após um deploy	Migration nova (ex.: `mfa_last_counter`) não aplicada no Neon; rode a seção "Mudança que altera o banco"
Login funciona e ao recarregar volta deslogado	Site chamando a API direto (sem proxy) ou `VITE_API_URL` errado: o cookie `SameSite=Strict` não é enviado
Erro 403 `FORBIDDEN` em POST/PUT	`CORS_ORIGIN` diferente do endereço real do site
`permission denied for table ...`	Script de papéis não rodou depois de uma migration nova, ou migração rodou com outro dono
502/504 na primeira visita	API dormindo (use o UptimeRobot)
Não chega e-mail, log `auth_email_delivery_failed` com timeout	`MAIL_TRANSPORT=smtp` no Render (SMTP bloqueado no grátis); use `http`
Não chega e-mail, log `auth_email_delivery_failed` com `mail_http_rejected` (script respondeu `{"ok":false}`)	`MAIL_HTTP_SECRET` diferente do `MAIL_SECRET` do script
Não chega e-mail, log com `mail_http_invalid_response`	O Google devolveu HTML: a implantação não está com acesso "Qualquer pessoa" ou a URL não é a `/exec`
Não chega e-mail, erro em **Execuções** do Apps Script	Cota diária (~100) estourada, ou o script foi editado sem publicar uma **Nova versão**
Resposta é uma página de login do Google em vez de JSON	Implantação com acesso diferente de **Qualquer pessoa**, ou URL de teste (`/dev`) em vez de `/exec`
Link de redefinir senha abre `localhost` ou dá erro	`APP_BASE_URL` no Render diferente do `URL_SITE`
`/termos` abre vazio no código-fonte ou mostra a landing	Redirect `/*` com `force = true` ou apontando para `/index.html`: deve ser `/spa.html` sem `force`
Prévia em rede social sem imagem ou com link errado	`SITE_URL` diferente do endereço real do site (veja o passo 6) ou cache antigo da rede social (use o Sharing Debugger para recarregar)

---

5. Como atualizar o site depois (corrigir erros e criar features)

Depois do deploy, **o GitHub é a fonte da verdade**: publicar = enviar código para a branch `main`. Netlify e Render observam o repositório e republicam sozinhos.

Fluxo do dia a dia

1. **Crie uma branch** a partir de `main`: `git switch -c feat/nome-da-mudanca` (ou `fix/...`).
2. **Desenvolva e teste localmente** (Docker + `npm run dev:all`). Antes de subir:

   ```
   npm run lint
   npm run test:strict
   npm --prefix frontend test
   npm run build
   ```

3. **Commit** com mensagem descritiva (sugestão: `feat(diario): adicionar X`, `fix(auth): corrigir Y`; não há validação automática).
4. **Abra um Pull Request** para `main`. O GitHub Actions (`.github/workflows/ci.yml`) roda auditoria, lint, testes e build. Só faça merge com o CI verde.
5. **Merge em `main`** → Render publica a API nova e Netlify publica o site novo (alguns minutos). Acompanhe em **Deploys** (Netlify) e **Events/Logs** (Render).
6. **Verifique em produção** com o checklist do passo 11 (os itens que a mudança tocou).

Mudança que altera o banco (migration)

O deploy do código **não** roda migrations; você roda:

1. Crie a migration em `backend/migrations/` e teste local.
2. **Antes do merge** (ou imediatamente antes de a API nova subir), aplique no Neon com o papel de migração:

   ```
   $env:DATABASE_URL="<URL_MIGRACAO>"
   npm run setup
   ```

3. Faça a migration **compatível com o código antigo** (adicionar coluna/tabela, sem remover nem renomear de uma vez). Assim a API antiga continua funcionando durante o deploy. Remoções ficam para uma migration posterior.
4. Como as tabelas novas pertencem ao `migration_user`, elas recebem as permissões do `app_user` automaticamente (configurado pelo script de papéis). Se aparecer `permission denied`, rode o script de papéis de novo.
5. Migrations já aplicadas **não se editam**: crie uma nova (veja `docs/MIGRATIONS.md`).

Conta de visitante (`POST /api/auth/demo`): a migration `1793600000000_add-guest-users` precisa rodar em produção (`npm run setup`, como acima) **antes** de subir o código novo. Ela é aditiva e segura com o código antigo no ar; sem ela, `PUT`/`DELETE /api/users/me` e MFA dão 500.

Variáveis novas ou segredos

Adicione primeiro no Render/Netlify, depois faça o merge. Segredos nunca entram no Git.

Erro urgente (hotfix)

1. Branch `fix/...` a partir de `main`, correção mínima, PR, CI verde, merge.
2. Se o site quebrou e você precisa voltar **agora**: Netlify → **Deploys** → versão anterior → **Publish deploy**; Render → **Events** → **Rollback** (ou reverter o commit no GitHub: `git revert <hash>` e push). Rollback **não** desfaz migrations.

Rotina de manutenção

- Olhe os **logs do Render** quando alguém reportar erro.
- Responda ao Dependabot/`npm audit`. O que importa para o site no ar é `npm audit --omit=dev` (hoje limpo); os avisos aceitos são só de ferramentas de dev (`minimatch@10` via `archunit`/`eslint`: um override global quebra o `archunit`; atualize quando eles aceitarem `minimatch@11`).
- Faça `backup-db.mjs` de vez em quando, pois o plano grátis do Neon tem retenção limitada de histórico.
- Se mudar o endereço do site, atualize `CORS_ORIGIN`, `APP_BASE_URL`, o widget do Turnstile, o `netlify.toml` e os 4 arquivos de SEO do passo 6 (`SITE_URL`, `robots.txt`, `sitemap.xml`, `llms.txt`).
- Criou uma página pública nova que deve aparecer no Google? Adicione-a em `PUBLIC_PAGES` (`frontend/src/seo/pages.ts`) para ser pré-renderizada, e no `sitemap.xml` e no `llms.txt`.

---

6. Pendências e riscos (honestidade)

- **Itens do `ISSUES.md` ligados a este plano:** #9 (papéis, concluído no passo 4), #24 (deploy: este plano vira o "provedor escolhido"), #26 (backup agendado) e #25 (monitoramento de erros) seguem pendentes.
- **Executado em 04/11/2026:** deploy no ar e checklist do passo 11 aprovado. Pendências encontradas em produção: #34 (`/login` logado dá 404), #35 (landing acessível logado) e #36 (canonical sem a barra final do redirect) em `docs/ISSUES.md`. O teste de bloqueio por IP atrás do proxy Netlify → Render (nota `TRUST_PROXY`) ainda não foi feito.
- **CSP:** `frontend/public/_headers` usa `connect-src 'self' https:`; com o proxy `/api` pode restringir só a `'self'` depois.
- **Dados de saúde (LGPD):** os itens #15, #30, #31 e #32 do backlog (criptografia em repouso, consentimento específico, minimização, retenção) estão adiados. Para um portfólio com dados fictícios, tudo bem; **não divulgue o site como serviço real para dados de saúde de terceiros** sem tratar isso.
- **SEO (pré-renderização):**
  - O domínio dos arquivos de SEO é `caloriesync-luiscorreia.netlify.app`; se o site no Netlify tiver outro nome, troque (passo 6).
  - O texto "gratuito" (landing e JSON-LD) precisa ser confirmado se o modelo do produto mudar.
  - Ao navegar dentro do app (sem recarregar), o `<title>` da aba não muda; ele só vem certo no carregamento direto, que é o que buscadores e redes sociais leem.

    - O `FAQPage` do JSON-LD fica só na landing, porque o Google exige que as perguntas estejam visíveis na página.
    - A Search Console do Google (enviar o `sitemap.xml`) só faz sentido com o site no ar; com `*.netlify.app` funciona, mas um domínio próprio rankeia e é lembrado melhor.
- **Testes instáveis:** em uma de três execuções, 2 testes de `frontend/src/tests/router.test.tsx` falharam por timeout ao carregar páginas lazy. Se o CI falhar só neles, rode de novo; se repetir, aumente o timeout do `findByRole`.
- **Disponibilidade:** plano grátis não tem SLA. É adequado a portfólio, não a produto com clientes.
- **Alternativa de menor complexidade** (se travar no proxy): hospedar front e API **juntos** no Render, com o Express servindo o `frontend/dist`. Isso exige mudar o backend (hoje ele não serve arquivos estáticos) e foge deste plano.

---

7. Arquitetura resumida (para você explicar em entrevista)

- Monorepo: `frontend/` (React 19, Vite, TanStack Query, Tailwind), `backend/` (Express + TypeScript, 4 camadas `routes → controllers → services → models`, SQL puro com `pg`, sem ORM), `shared/` (schemas Zod e tipos compartilhados).
- Segurança: JWT em cookie HttpOnly `SameSite=Strict`, verificação de e-mail, bloqueio de conta, rate limit em Postgres, captcha opcional, MFA (TOTP) obrigatório para admins, RBAC com papel lido do banco a cada requisição, CORS estrito e papéis de banco com menor privilégio.
- Qualidade: testes (backend com `node:test`, front com Vitest/RTL/MSW), CI no GitHub Actions, Dockerfile.
- Mais detalhes: `docs/ARCHITECTURE.md`, `docs/DECISOES.md`, `docs/DB_ROLES.md`, `docs/EMAIL_SETUP.md`, `docs/ADMIN.md`, `docs/MIGRATIONS.md` e o grafo em `.ua/knowledge-graph.json`.

Fontes consultadas (02/11/2026)

- Render: Deploy for Free (spin-down de 15 min, 750 h, sem disco, Postgres grátis expira em 30 dias)
- Render: Postgres grátis expira em 30 dias
- Neon: limites do plano grátis e anúncio de 1 GB por projeto
- Render: plano grátis bloqueia saída SMTP (25, 465, 587) desde 26/09/2025
- Apps Script: Web Apps, MailApp e cotas
- Brevo: domínios gratuitos (gmail.com) não podem ser autenticados
- Netlify: rewrites e proxies

Roadmap de Desenvolvimento (ROADMAP.md)

Este documento registra o planejamento estratégico e a evolução do **CalorieSync**, organizando as fases concluídas e os próximos ciclos de desenvolvimento.

---

1. Diretrizes de Execução

1. **Desenvolvimento Direto e Focado:** Implementações alinhadas aos requisitos e fórmulas em docs/SPEC.md.
2. **Qualidade e Estabilidade:** Toda alteração é acompanhada de verificação via `npm run test`, `npm --prefix frontend test`, `npm run lint` e `npm run build`.
3. **Padrão Brutalista:** Respeito integral aos tokens semânticos e regras de interface em docs/CORES.md.

---

2. Fases Concluídas ✅

- **Fase 0: Fundação e Qualidade:** Monorepo configurado, banco de dados isolado para testes (`contador_calorias_test`), migrações com `node-pg-migrate`, ESLint e CI.
- **Fase 1: Motor Nutricional e Metas:** Fórmulas de TMB (Mifflin-St Jeor / Katch-McArdle), TDEE, pisos calóricos, divisão de macronutrientes e vigência temporal de metas por data.
- **Fase 2: Diário e Estatísticas:** Visão diária, consumo vs. metas, agregações analíticas (`LATERAL JOIN`), médias inteligentes e visualização de macronutrientes.
- **Fase 3: Catálogo de Alimentos em Escala:** Busca tolerante com trigram (`pg_trgm`) e `unaccent`, catálogo protegido e importação de tabelas nutricionais (TACO).
- **Fase 4: Robustez e Performance:** Roteamento com React Router 7, code-splitting por rota com `React.lazy`, compressão HTTP, cache, inserção multi-row e testes de integração de autorização multiusuário.
- **Fase 4.5: Produtividade do Diário:** Navegação histórica por data, cópia rápida de refeições, histórico de recentes/favoritos e refeições prontas/combos salvos.
- **Fase 5: Design Brutalista e Refatoração:** Migração completa para tokens semânticos e CSS variables (Neo-Brutalismo de Tinta), conformidade de acessibilidade (WCAG AA), formulários com `react-hook-form` + Zod e decomposição de componentes.
- **Fase 5.5: Pacote Compartilhado (`shared`):** Contratos DTO, schemas Zod e cálculos matemáticos compartilhados como fonte única entre backend e frontend.
- **Fase 6 (parcial): Autenticação por e-mail:** Confirmação de cadastro por código de 6 dígitos (BL-12) e recuperação de senha por link de uso único (BL-05), com hard gate no login, revogação por `token_version`, Mailpit/Mailer plugável e telas públicas de verificação e redefinição.
- **Fase 6.5: Painel Administrativo do Catálogo e Curadoria (D-22 / B1–B12):** RBAC estrito via banco (`users.role`), auditoria transacional atômica (`admin_audit_log`), rotas protegidas sob `/api/admin/foods`, CLI de gestão de administradores (`admin:promote`, `admin:demote`, `admin:list`), ferramentas de curadoria reproduzíveis (export CSV com heurísticas de

duplicatas, apply com simulação/efetivação, import estruturado de curados), proteção de alimentos curados contra reativação e sobreposição na TACO (B10), tela brutalista `/admin/alimentos` e `AdminRoute`.
- [x] **UX: navegação mobile e renomeação para Diário:** tela "Dashboard" renomeada para "Diário"; barra inferior mobile com auto-ocultar; tela `/conta`; `SegmentedNav` única para submenus; swipe entre os destinos da barra inferior com arrasto animado; módulo único de destinos (`frontend/src/navigation/`) e host global do modal de Nova Refeição.

---

3. Próximos Ciclos de Desenvolvimento ⏳

⏳ Fase 6: Autenticação Avançada e Recuperação de Acesso

- [x] Confirmação de e-mail em duas etapas (código OTP de 6 dígitos, BL-12).
- [x] Fluxo de recuperação de senha ("Esqueci minha senha") com envio de e-mail e token temporário de 30 minutos (BL-05).
- [ ] Login social (OAuth2 via Google e Apple) integrado à autenticação do backend.
- [ ] Proteção contra automação maliciosa nas rotas públicas de autenticação (ex: Cloudflare Turnstile).

⏳ Fase 7: Adequação à LGPD e Privacidade

- [x] Exportação completa e portabilidade dos dados do usuário em JSON.
- [x] Consentimento explícito no cadastro (`terms_accepted_at`) e termos legais.
- [ ] Rotina agendada para expurgo seguro de contas inativas há mais de 2 anos.

⏳ Fase 8: Sessão Segura e Observabilidade em Produção

- [ ] Migração de JWT armazenado no cliente para cookies seguros `HttpOnly` com Refresh Tokens rotacionados em banco.
- [x] Habilitação de `trust proxy` e rate limiting por IP + e-mail.
- [x] Content Security Policy (CSP) e cabeçalhos de segurança via Helmet.
- [x] Logging HTTP estruturado com correlation ID (`X-Request-Id`).
- [ ] Monitoramento e rastreamento de erros em tempo real (ex: Sentry).
- [ ] Pipeline automatizado de deploy contínuo (CI/CD).

---

4. Backlog de Funcionalidades Futuras

- Aplicativo mobile dedicado em React Native / Expo (consumindo o pacote `@caloriesync/shared`).
- Registro diário de ingestão de água com meta personalizada e progresso visual.
- Leitor de código de barras integrado com a API do Open Food Facts.
- Distribuição de metas nutricionais segmentadas por tipo de refeição (ex: café, almoço, jantar).